23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No : A5

Team Members :

Roll No	Student Name
AM.SC.U4CSE25057	VAISHNAV VENU
AM.SC.U4CSE25060	PRAFUL SINGH
AM.SC.U4CSE25063	C LASYA
AM.SC.U4CSE25066	PRATHIJNA

Project Title : **Hotel Room Booking and Hospitality Management System**

CONTENTS

Sl. No	Chapter	Page No
1	Abstract	
2	Introduction	
3	Requirements (Functional & Non-functional)	
4	UI Design	
5	Database Design	
6	Database Selection & Justification	
7	Implementation	
8	Testing & Validation	
9	Results & Screenshots	
10	Challenges faced & solutions	
11	Conclusion & future enhancements	
	References	

1. Abstract

The **Hotel Room Booking and Hospitality Management System** demands synchronized coordination across room inventory allocation, guest tracking, amenity ful-fillment, point-of-sale invoicing, and housekeeping workflows. Traditional, decoupled spreadsheets or primitive database architectures frequently result in data redundancy, double-booking race conditions, delayed housekeeping room turnover, and financial leakage from untracked ancillary services.

To solve these operational challenges, this project presents an enterprise three-tier Hotel Room Booking and Hospitality Management System centered on a normalized, ACID-compliant **PostgreSQL** database engine interfaced through a Python Flask REST application tier and role-specific web dashboards. The database architecture employs a 3NF/BCNF relational model bound by engine-level domain constraints (CHECK, foreign key cascaded updates/deletions, non-overlapping temporal date constraints).

Autonomous database behaviour is driven by custom PL/pgSQL routines:

1. A date-range intersection validation algorithm that mathematically prevents room overbooking.
2. Cascading event triggers that monitor checkout states to immediately schedule housekeeping tasks.
3. Atomic checkout procedures that aggregate multi-night tariffs, in-room amenities, discounts, and luxury tax calculations into immutable bills.
4. PostgreSQL JSONB Change-Data-Capture (CDC) logging for tamper-resistant auditing.

Performance optimization is sustained through partial indexes, composite indices, and Materialized Views caching high-order financial metrics. This architecture ensures transactional isolation, zero financial discrepancy, high-throughput query performance, and consistent relational integrity.

This project addresses the need for a centralized system to manage hotel reservations, guest services, room availability, and billing. Its objective is to make daily hotel operations more efficient and give guests and staff clear, coordinated access to the information and tools they need. The proposed solution is a web-based hotel management system with guest and reception portals, supported by a PostgreSQL database. Major database features include linked records for customers, rooms, reservations, service and food orders, housekeeping, bills, expenses, offers, and lost-and-found reports, with constraints and triggers that support booking integrity and routine workflows. The expected outcome is more reliable reservation and room management, clearer folio tracking, and faster coordination between guests and hotel staff.

2. Introduction

Hotels coordinate many activities every day: assigning rooms, recording guest details, managing reservations, responding to service requests, maintaining room readiness, and preparing bills. These activities are closely connected. A change to a guest's stay dates can affect room availability and charges; a completed food or amenity order can affect the guest's folio; and checkout can trigger housekeeping work before the room is ready for another guest. Managing these processes requires accurate, up-to-date information that can be accessed by the appropriate staff and guests.

In a hotel that relies on paper records, spreadsheets, or separate applications for these tasks, information can become fragmented. Staff may need to enter the same details in multiple

places, making records harder to keep consistent. A reservation update might not be reflected promptly in room details, service records, or billing. Tracking which orders have been completed and which charges belong to a particular stay can also require manual effort. These limitations can slow routine work and make it more difficult to provide guests with a clear account of their reservations and expenses.

The Hotel Management System DBMS project addresses this problem with a web-based application supported by a PostgreSQL relational database. It provides separate portals for guests and hotel staff. Guests can create accounts, browse room types, make reservations for selected dates, place food orders, request amenities or housekeeping, review their stay history, and view folio details. Reception staff can review reservations and room information, manage service requests, monitor housekeeping tasks, and access detailed stay charges. This division gives users access to the functions relevant to their role while keeping the underlying information connected.

The database is central to the system. It stores information about customers, staff, rooms, reservations, services, service requests, bills, housekeeping tasks, expenses, offers, and lost-and-found reports. Relationships between these records associate each reservation with a guest and a room, and associate orders, expenses, and bills with the correct stay. Database constraints help maintain valid records, such as requiring checkout to occur after check-in. The application also checks room availability for requested dates to prevent overlapping reservations for the same room. Database triggers support operational workflows, including creating housekeeping tasks when a room enters cleaning status.

The system is designed to keep reservation and billing information useful throughout a guest's stay. Guests can review itemized food orders, amenity and facility requests, and other recorded expenses alongside folio totals. They can also request date changes before checkout; the system validates the requested dates against reservation rules and room availability. On the staff side, reception can view corresponding stay and expense details, helping keep guest-facing and operational records aligned. Room inventory is organized by room type, so guests can choose a suitable category while reception can inspect individual rooms.

By bringing these processes into one database application, the project aims to reduce duplicated recordkeeping and make hotel operations easier to follow. The expected outcome is a more consistent way to manage reservations, guest requests, room readiness, and billing, supported by a shared source of data for guests and staff.

A hotel property (**Crowne Plaza**) encompasses multiple interdependent operational pillars:

1. **Room Inventory & Lifecycle Management**: Tracking room categories (Deluxe, Suite, Economy), dynamic pricing per night, and physical states (*Available, Occupied, Cleaning, Maintenance*).
2. **Guest Registration & Stay Tracking**: Capturing personal credentials, stay histories, contact details, and special preferences.
3. **Reservation & Scheduling Lifecycle**: Managing booking confirmations, check-ins, extensions, and cancellations across fluctuating dates.
4. **On-Demand Hotel Amenities & Service Ful-fillment**: Handling ancillary guest requests (breakfast in bed, spa packages, laundry, extra rollaway beds).
5. **Point-of-Sale (POS) Billing & Invoicing**: Accurately aggregating room night charges, amenity consumption, tax computation (5%), and promotional discounts into an itemized invoice.

6. **Housekeeping & Facility Maintenance**: Coordinating rapid room turnaround between a guest's checkout and the next guest's arrival.
7. **Multi-Role Administrative Governance**: Delegating operational duties across **Guests** (self-service booking), **Receptionists** (front-desk check-in/out & service dispatch)

3. Requirements (Functional & Non-functional)

Functional Requirements:

1. **Authentication and roles:** The system shall allow guests, reception staff, and administrators to sign in and access functions appropriate to their roles. Guests shall be able to create accounts and manage their own reservations.
2. **Room inventory:** Staff shall be able to view room types and individual rooms, including room numbers, rates, facilities, and availability. Guests shall browse room types and available options.
3. **Reservations:** Guests or authorized staff shall be able to create reservations with guest details, room type, check-in and check-out dates, and guest count. The system shall prevent invalid dates and overlapping reservations for the same room.
4. **Stay updates:** Guests shall be able to request changes to reservation dates before checkout. The system shall validate the dates and room availability, then reflect accepted changes in reservation and folio information.
5. **Guest services and food orders:** Guests shall be able to place food orders and submit amenity or housekeeping requests for an active reservation. Staff shall be able to view and update the status of service requests.
6. **Folios and billing:** The system shall associate room charges, completed service and food orders, additional expenses, taxes, discounts, and payments with the correct reservation. Guests and staff shall be able to view stay charges and itemized folio details.
7. **Offers:** The system shall display relevant offers and eligibility information. Where required, guests shall be able to submit supporting evidence, such as a student ID, for staff review.
8. **Housekeeping:** The system shall record housekeeping tasks, allow staff to update their status, and support room readiness workflows following checkout.
9. **Lost and found:** Guests and staff shall be able to create and track lost-and-found reports and their updates.
10. **Search and reports:** Authorized staff shall be able to search reservations and guests, review operational records, and access summaries such as room status, service activity, bills, and revenue.

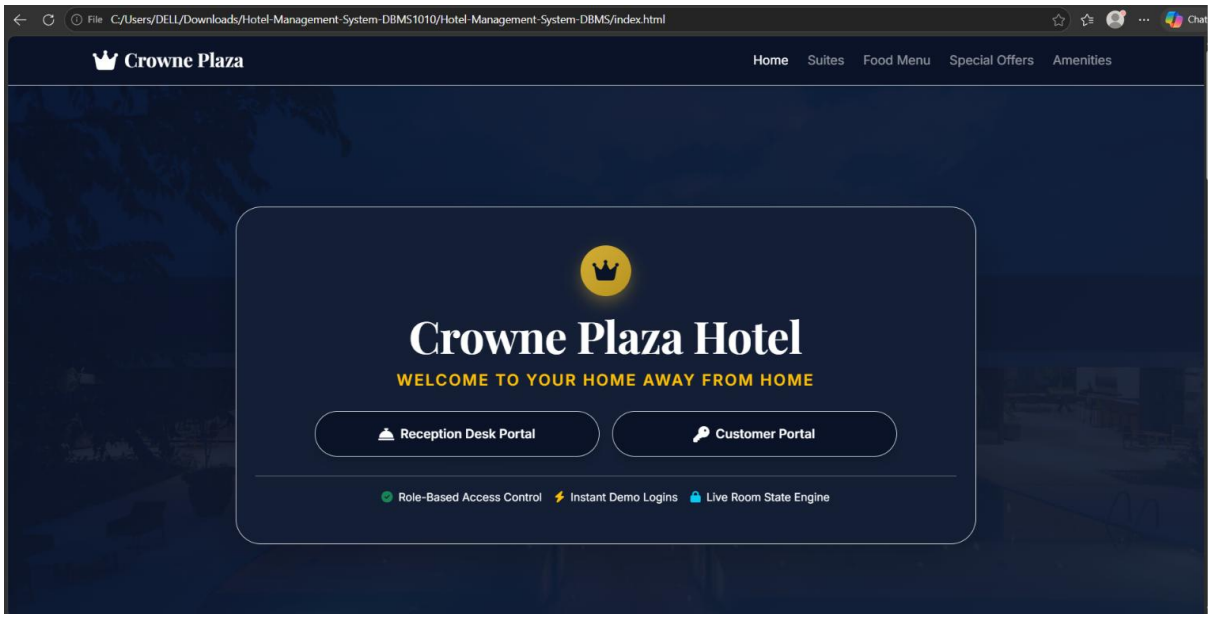
Non-functional Requirements

1. **Security:** The system shall authenticate users, restrict access according to role, protect guest records from access by other guests, and store passwords securely.
2. **Data integrity:** The database shall use relationships, constraints, and transactions to keep reservations, orders, expenses, and bills consistent and prevent invalid or conflicting records.
3. **Performance:** Common actions, including searching reservations, loading room information, and viewing folios, should complete promptly under normal hotel usage.
4. **Reliability:** The system shall handle failed database operations without leaving partially saved reservation or billing records and should provide clear error feedback.
5. **Usability:** Guest and staff portals shall present clear navigation, readable status information, and validation messages, and shall work on common desktop and mobile browsers.

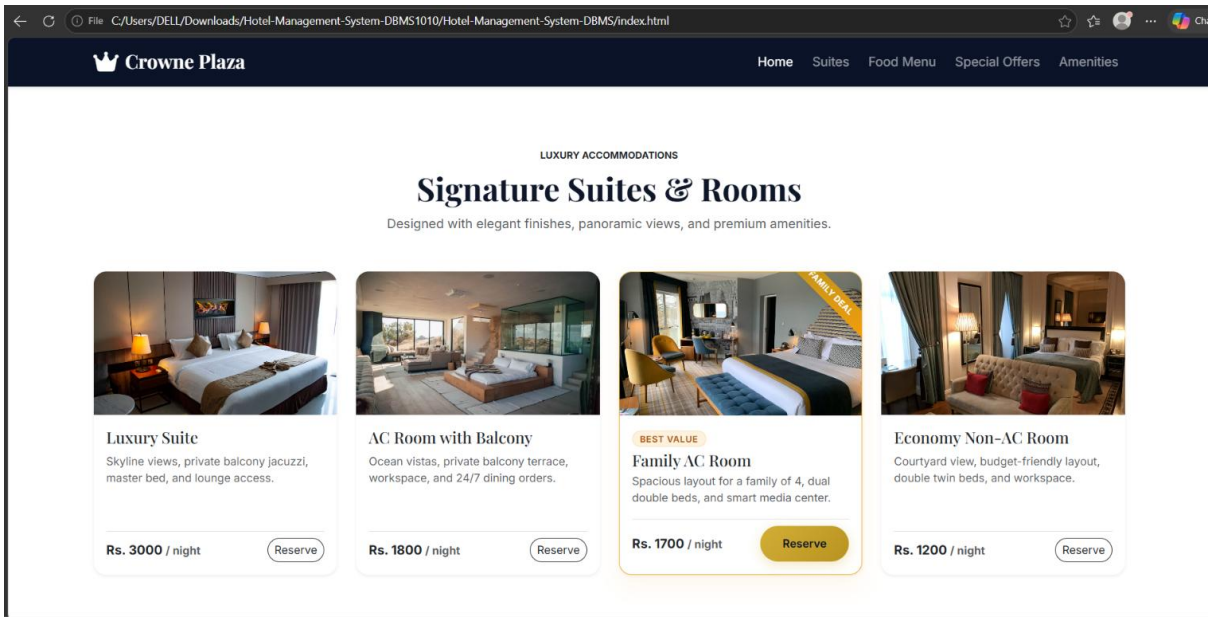
6. **Maintainability and scalability:** The application shall use a structured database and modular services so new rooms, services, users, and operational features can be added without redesigning the entire system.

4. UI Design

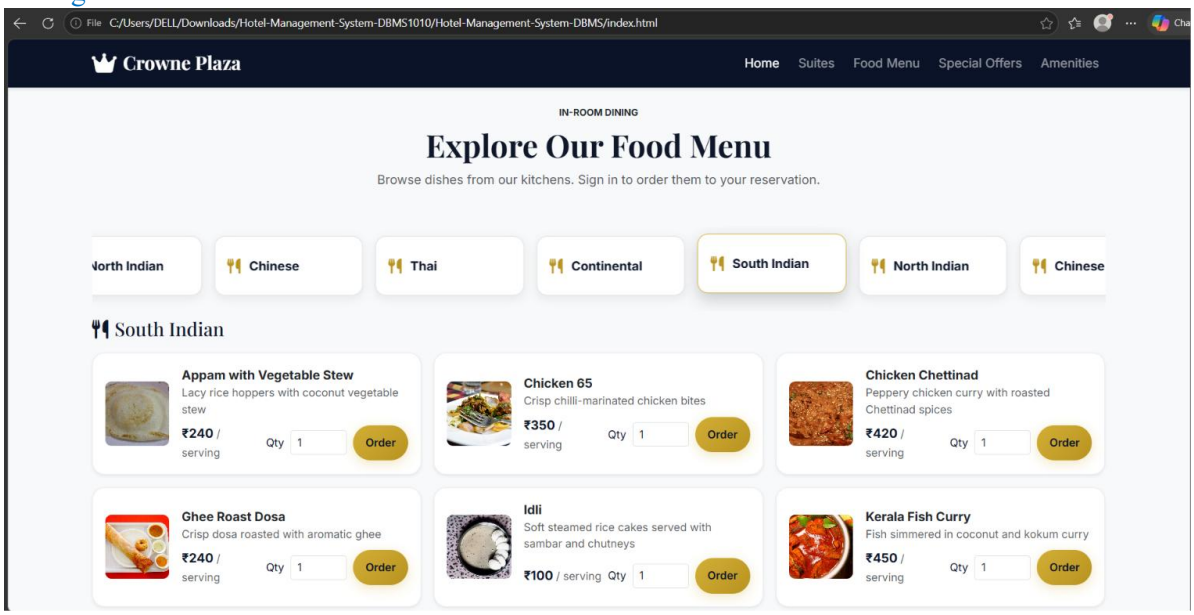
Home Screen:



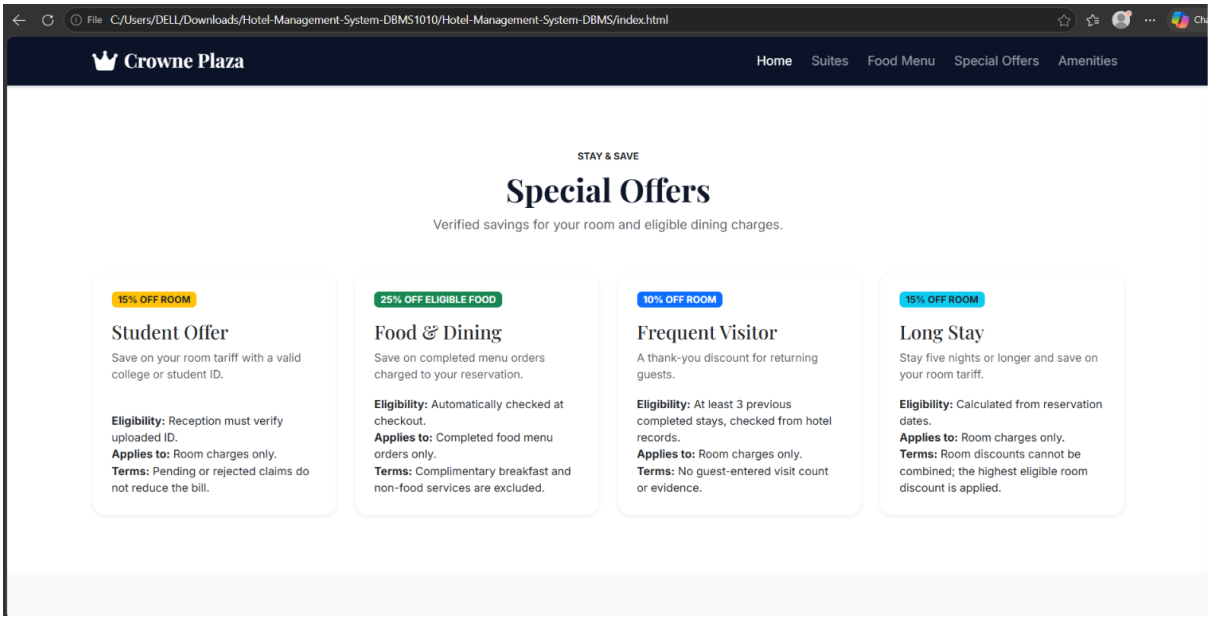
Suites:



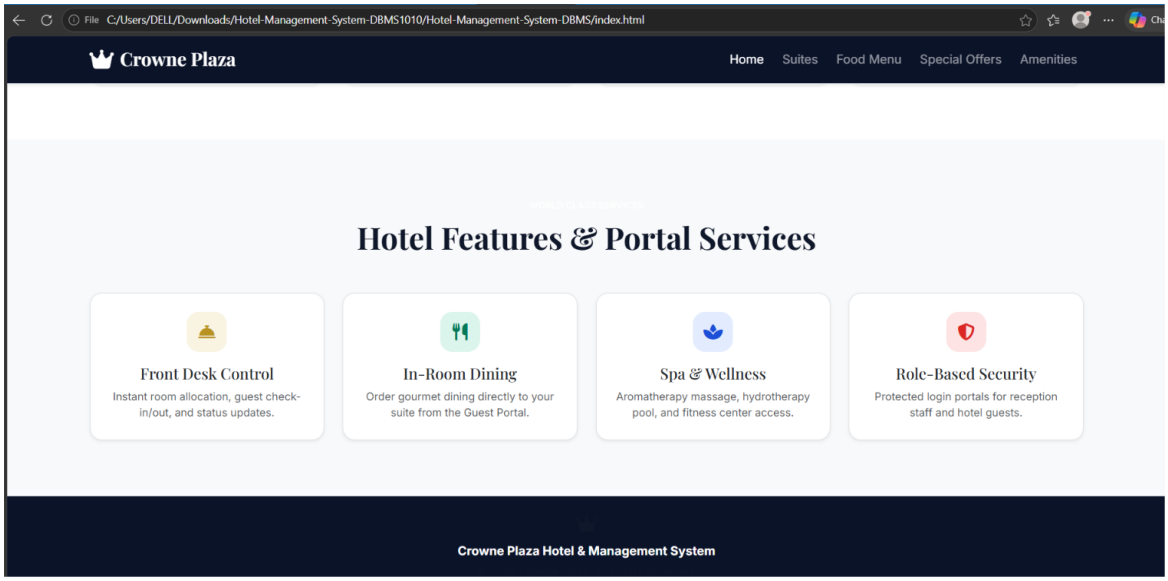
Dining/ Food menu:



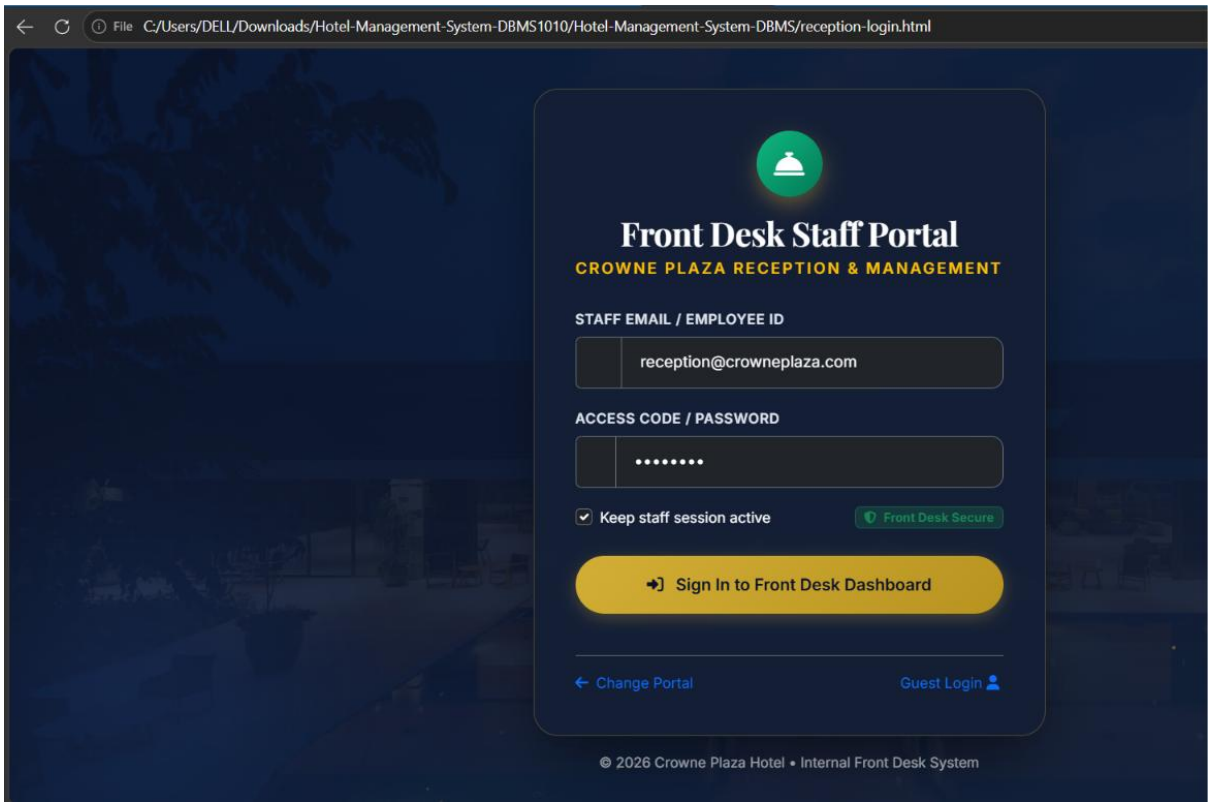
Special Offers:



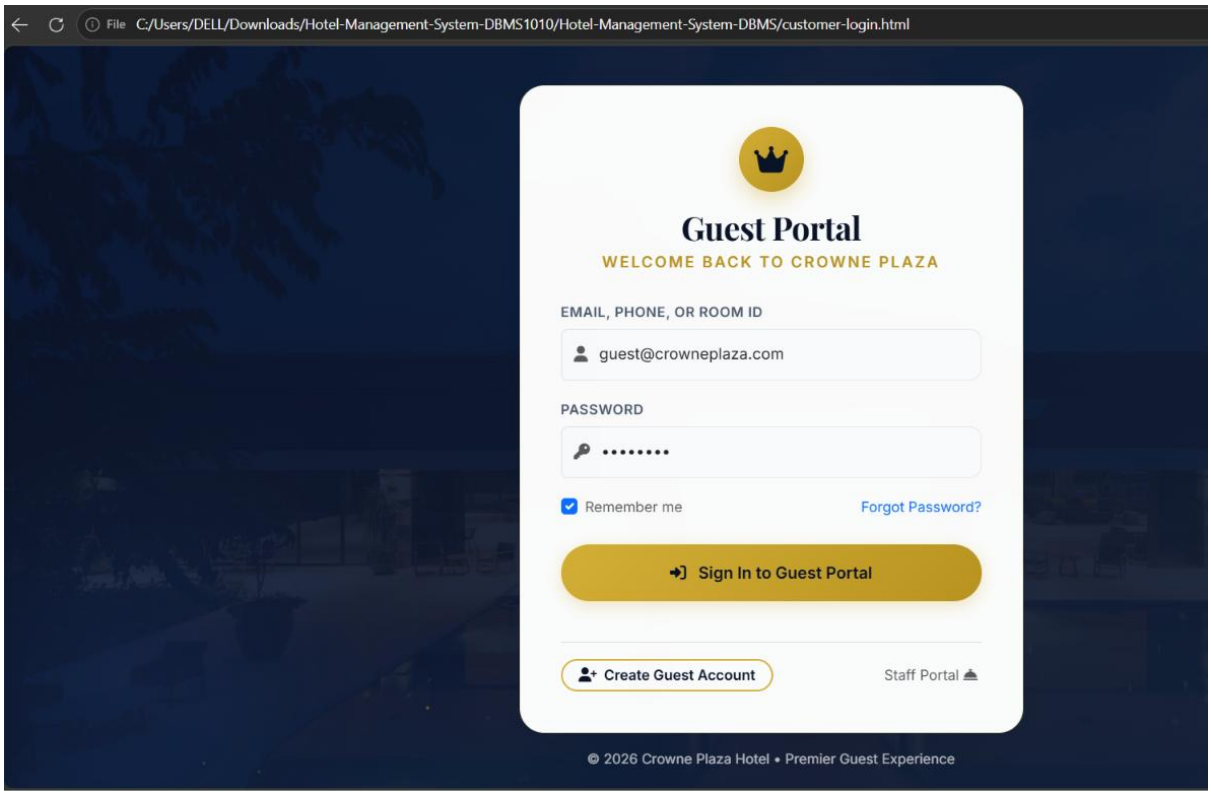
Amenities:



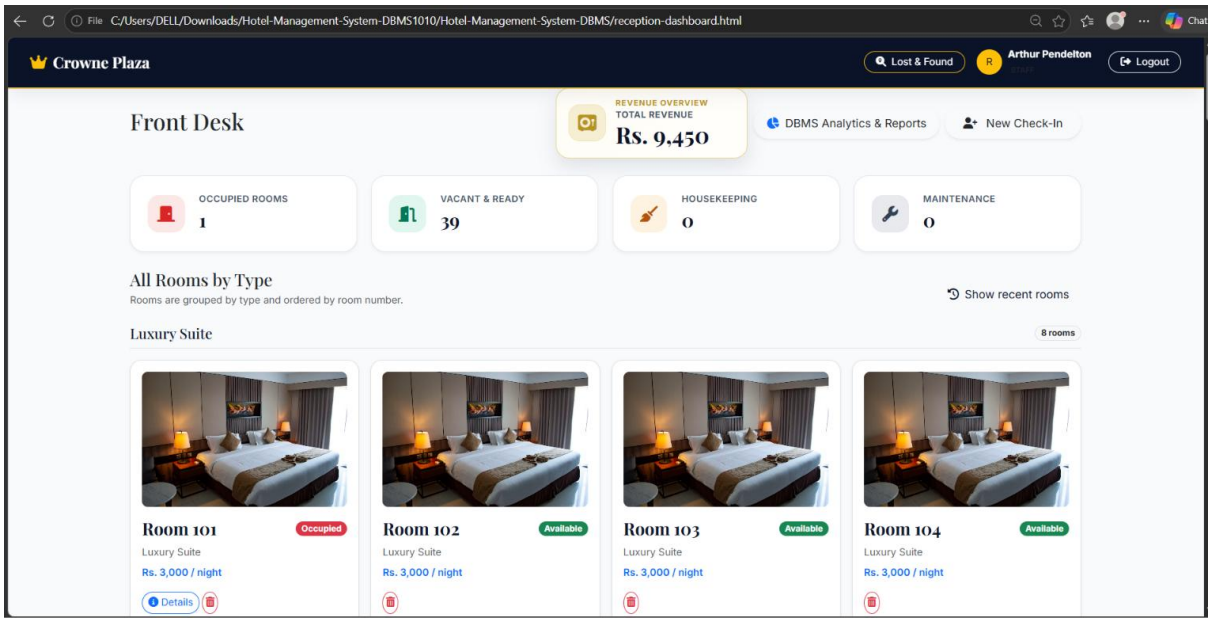
Reception Login:

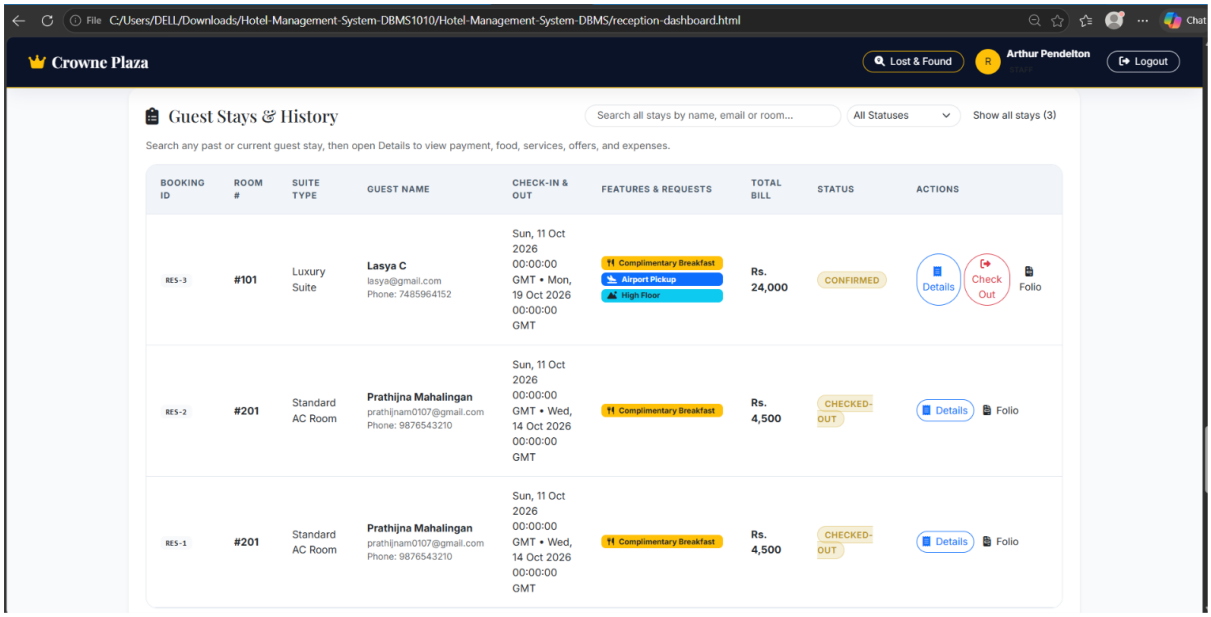


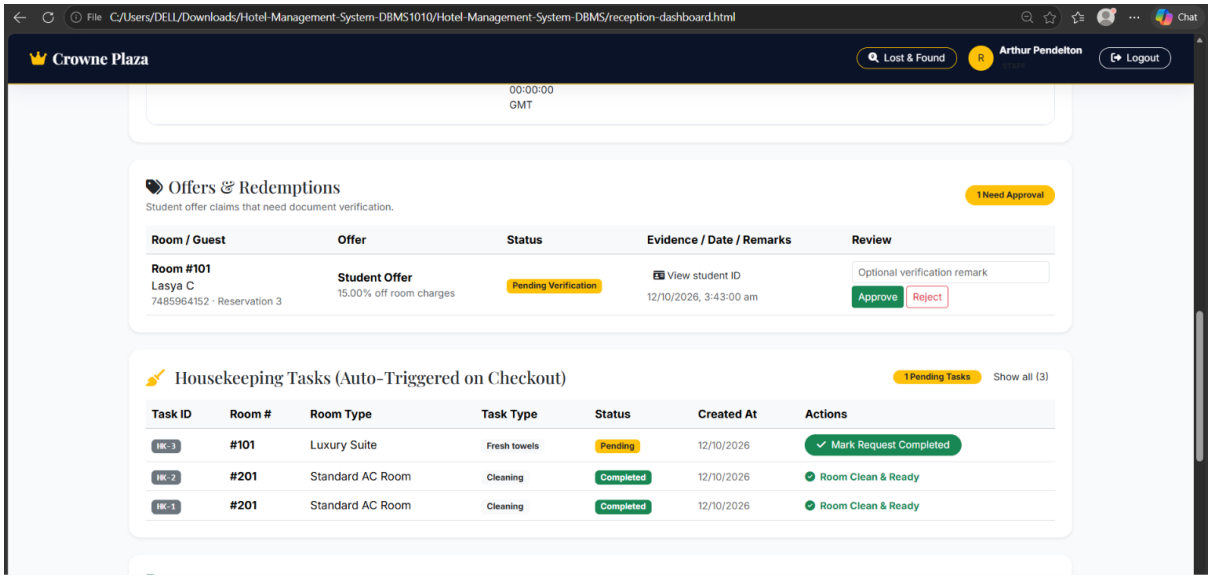
Guest login:

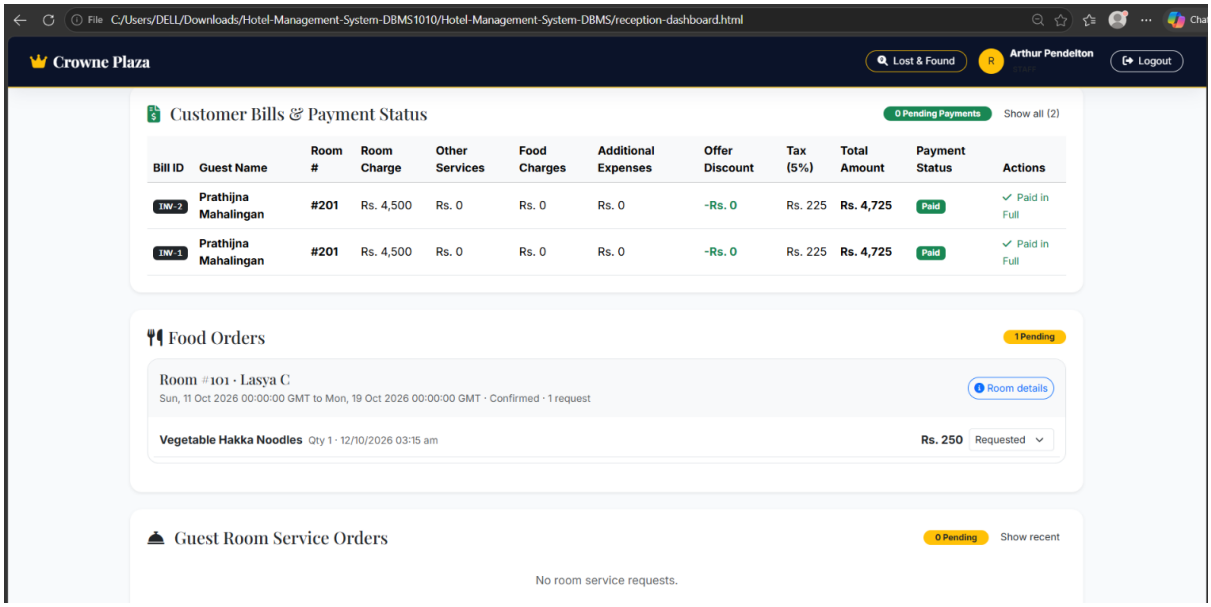


Front desk (Room status):

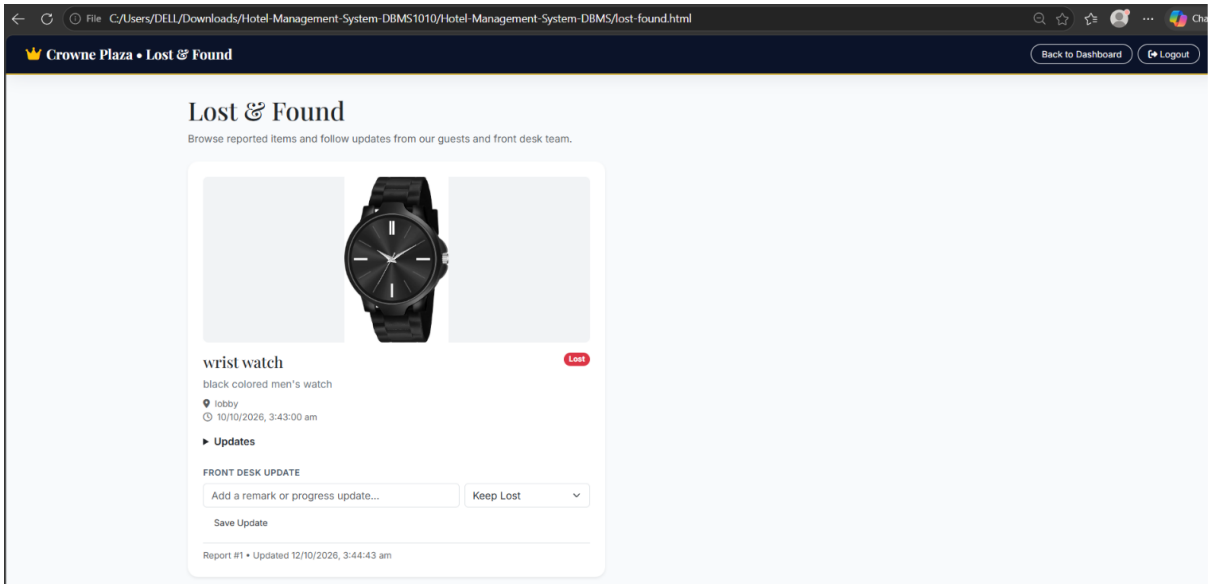




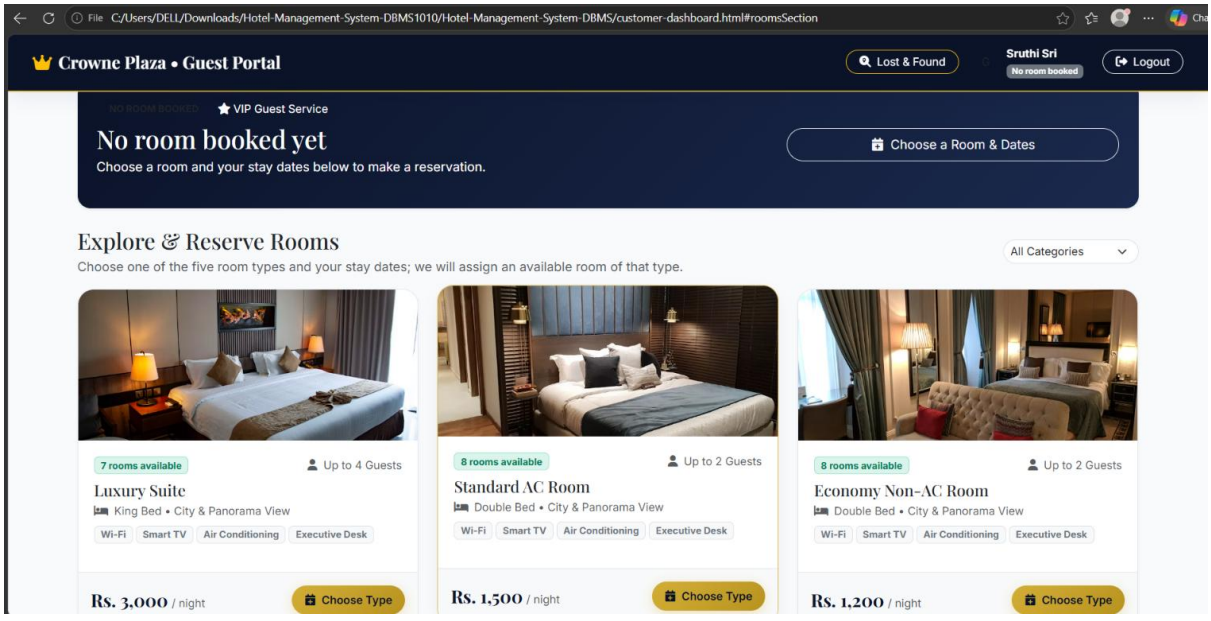


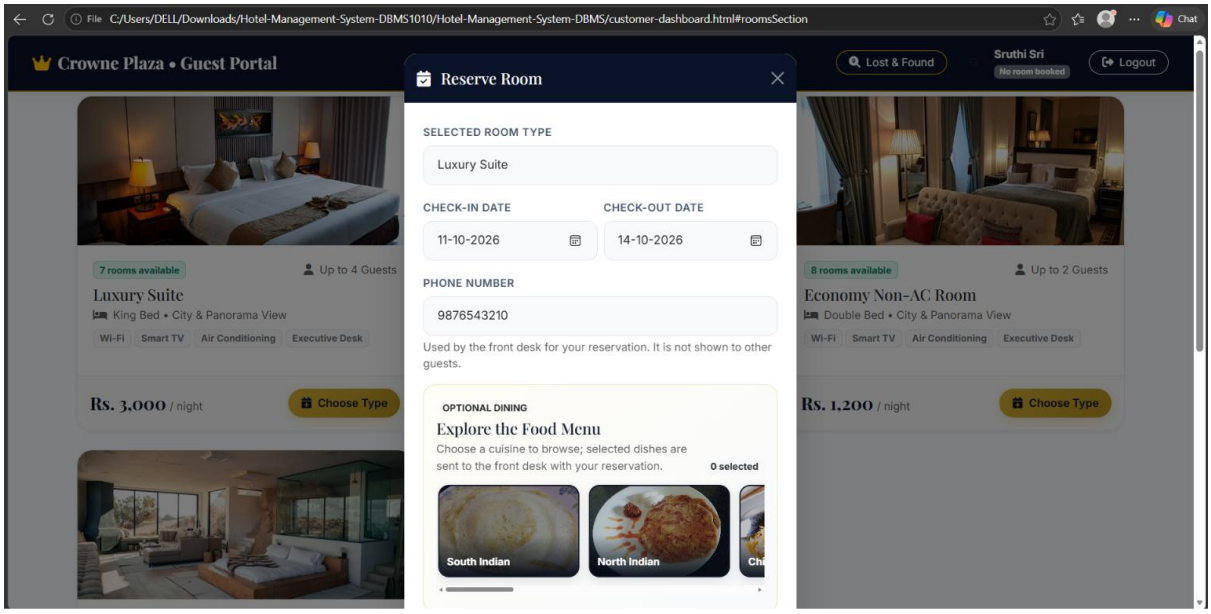


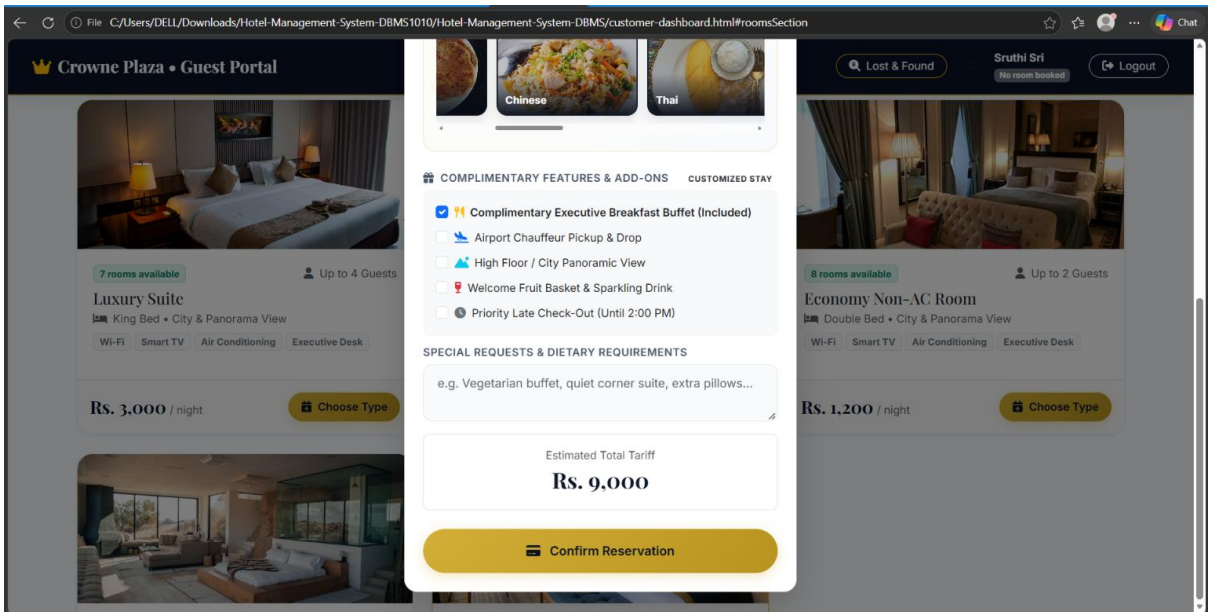
Lost and Found:

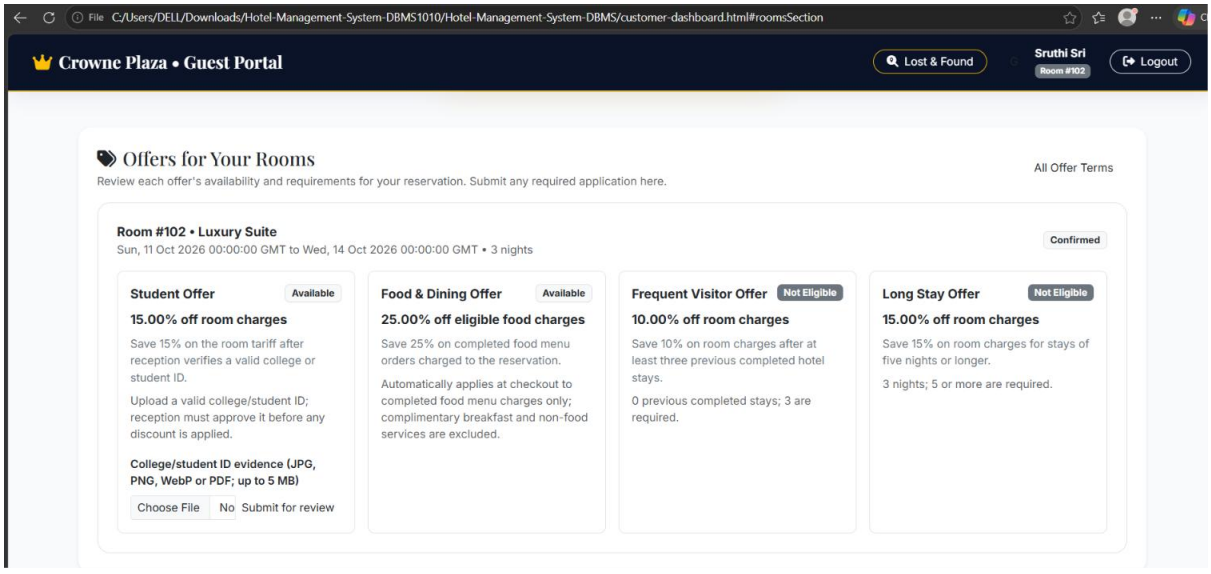


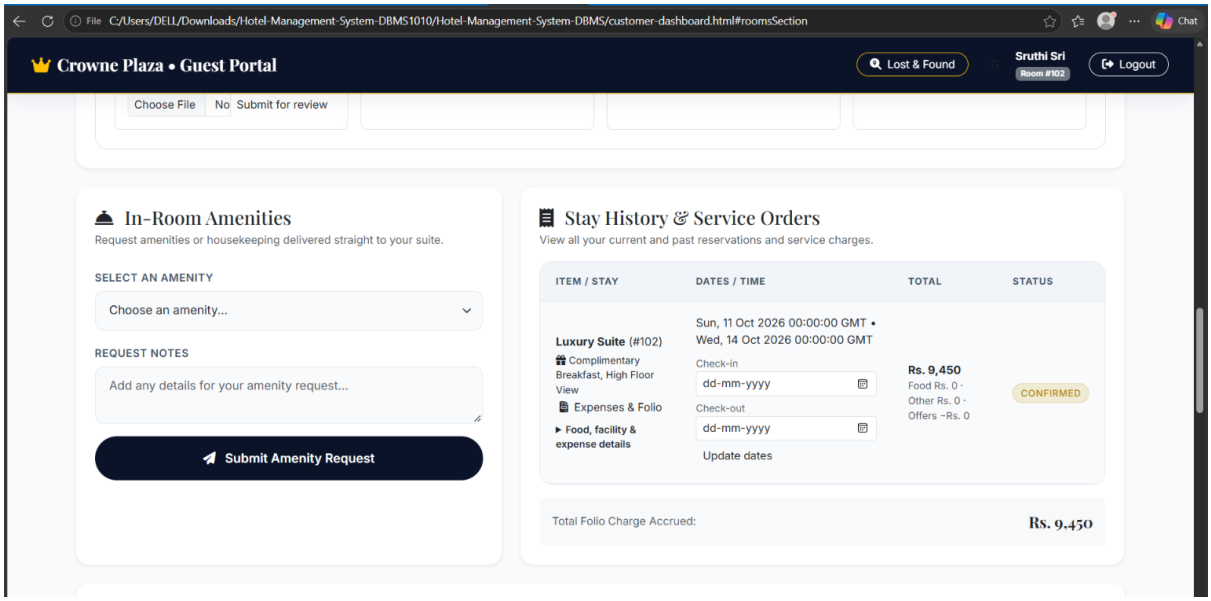
Guest portal:

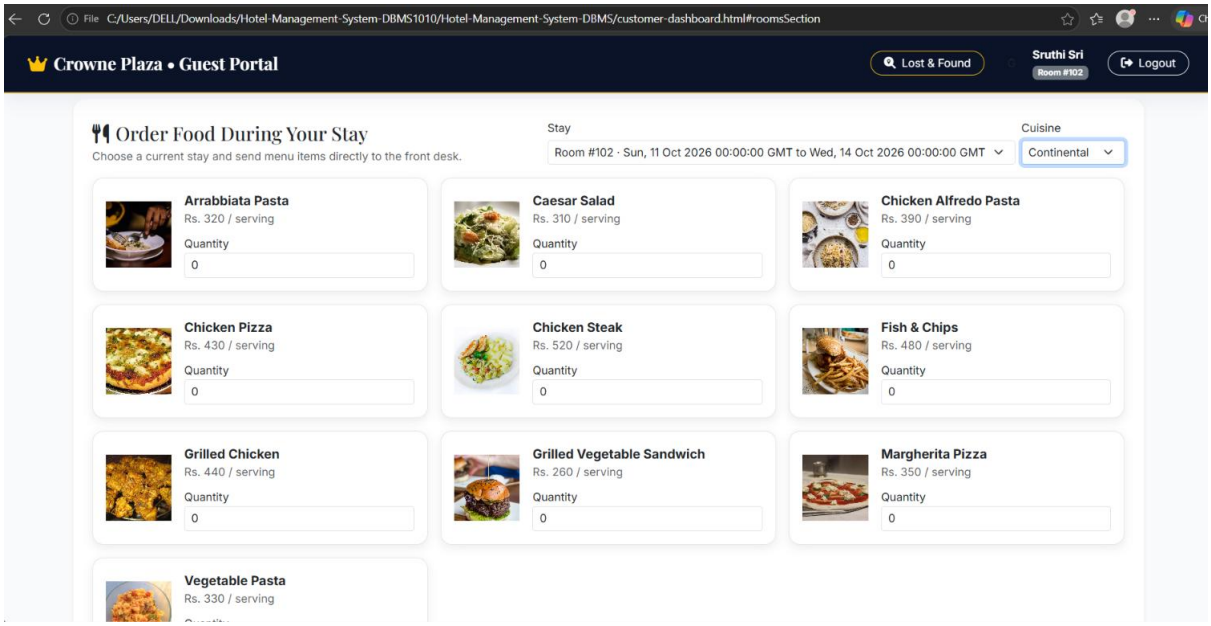


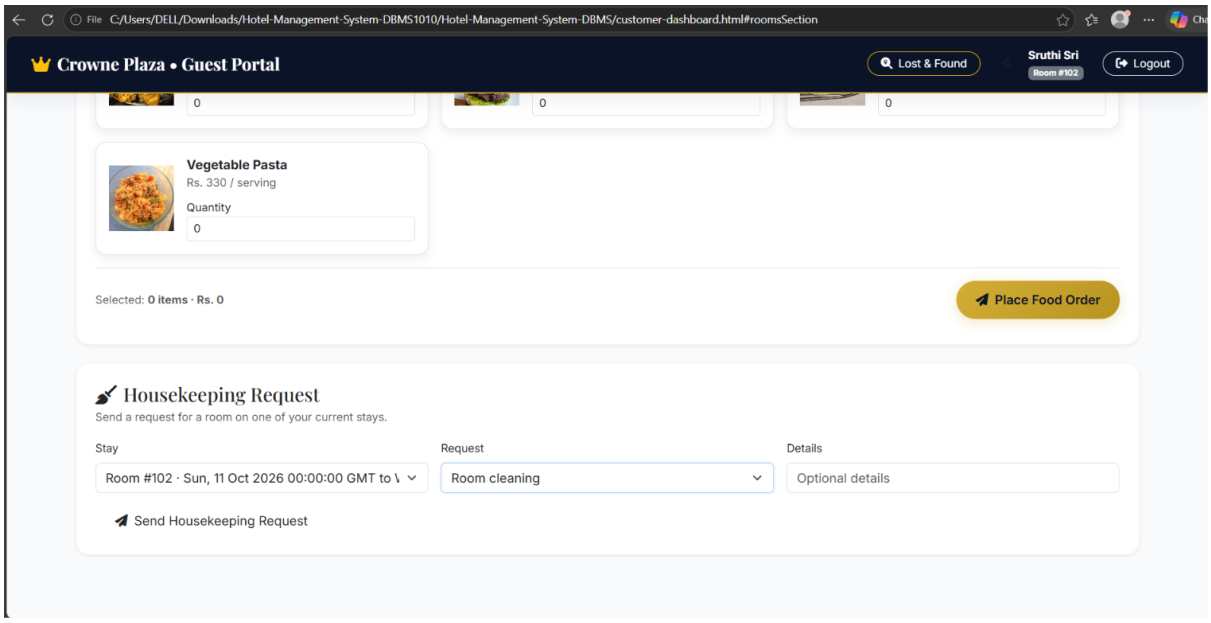

File C:/Users/DELL/Downloads/Hotel-Management-System-DBMS1010/Hotel-Management-System-DBMS/customer-dashboard.html#roomsSection
Chat
Crowne Plaza • Guest Portal
Lost & Found
Sruthi Sri
No room booked
Logout
Reserve Room
SELECTED ROOM TYPE
Luxury Suite
CHECK-IN DATE
11-10-2026
CHECK-OUT DATE
14-10-2026
PHONE NUMBER
9876543210
Used by the front desk for your reservation. It is not shown to other guests.
OPTIONAL DINING
Explore the Food Menu
Choose a cuisine to browse; selected dishes are sent to the front desk with your reservation.
0 selected
South Indian
North Indian
7 rooms available
Up to 4 Guests
Luxury Suite
King Bed • City & Panorama View
Wi-Fi
Smart TV
Air Conditioning
Executive Desk
Rs. 3,000 / night
Choose Type
8 rooms available
Up to 2 Guests
Economy Non-AC Room
Double Bed • City & Panorama View
Rs. 1,200 / night
Choose Type


File C:/Users/DELL/Downloads/Hotel-Management-System-DBMS1010/Hotel-Management-System-DBMS/customer-dashboard.html#roomsSection
Chat
Crowne Plaza • Guest Portal
Lost & Found
Sruthi Sri
No room booked
Logout
Chinese
Thai
COMPLIMENTARY FEATURES & ADD-ONS
CUSTOMIZED STAY
Complimentary Executive Breakfast Buffet (Included)
Airport Chauffeur Pickup & Drop
High Floor / City Panoramic View
Welcome Fruit Basket & Sparkling Drink
Priority Late Check-Out (Until 2:00 PM)
SPECIAL REQUESTS & DIETARY REQUIREMENTS
e.g. Vegetarian buffet, quiet corner suite, extra pillows...
Estimated Total Tariff
Rs. 9,000
Confirm Reservation
7 rooms available
Up to 4 Guests
Luxury Suite
King Bed • City & Panorama View
Wi-Fi
Smart TV
Air Conditioning
Executive Desk
Rs. 3,000 / night
Choose Type
8 rooms available
Up to 2 Guests
Economy Non-AC Room
Double Bed • City & Panorama View
Rs. 1,200 / night
Choose Type

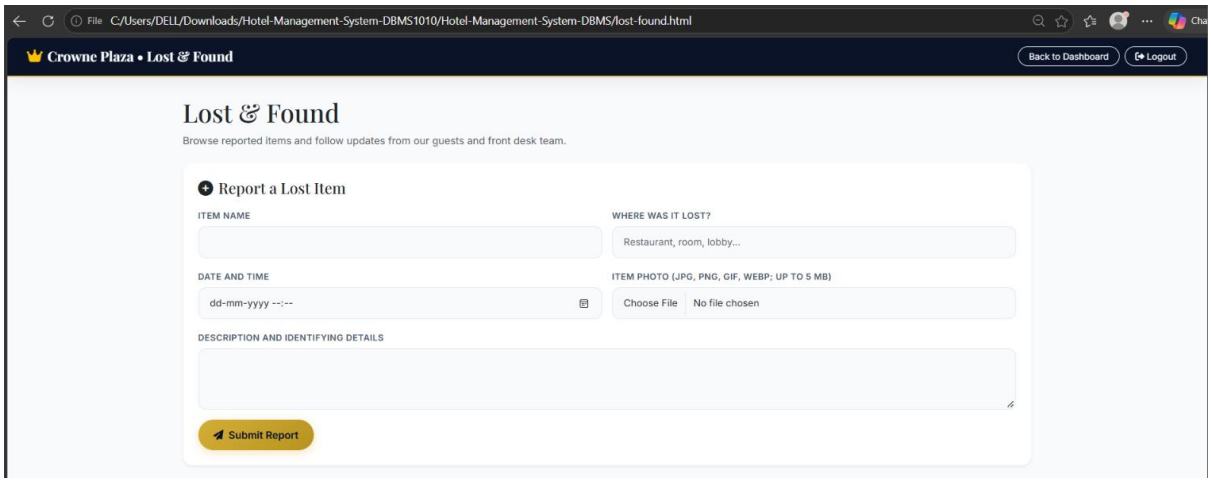

File C:/Users/DELL/Downloads/Hotel-Management-System-DBMS1010/Hotel-Management-System-DBMS/customer-dashboard.html#roomsSection
Crowne Plaza • Guest Portal
Lost & Found
Sruthi Sri
Room #102
Logout
Offers for Your Rooms
Review each offer's availability and requirements for your reservation. Submit any required application here.
All Offer Terms
Room #102 • Luxury Suite
Sun, 11 Oct 2026 00:00:00 GMT to Wed, 14 Oct 2026 00:00:00 GMT • 3 nights
Confirmed
Student Offer
Available
15.00% off room charges
Save 15% on the room tariff after reception verifies a valid college or student ID.
Upload a valid college/student ID; reception must approve it before any discount is applied.
College/student ID evidence (JPG, PNG, WebP or PDF; up to 5 MB)
Choose File
No
Submit for review
Food & Dining Offer
Available
25.00% off eligible food charges
Save 25% on completed food menu orders charged to the reservation.
Automatically applies at checkout to completed food menu charges only; complimentary breakfast and non-food services are excluded.
Frequent Visitor Offer
Not Eligible
10.00% off room charges
Save 10% on room charges after at least three previous completed hotel stays.
0 previous completed stays; 3 are required.
Long Stay Offer
Not Eligible
15.00% off room charges
Save 15% on room charges for stays of five nights or longer.
3 nights; 5 or more are required.


File C:/Users/DELL/Downloads/Hotel-Management-System-DBMS1010/Hotel-Management-System-DBMS/customer-dashboard.html#roomsSection
Crowne Plaza • Guest Portal
Lost & Found
Sruthi Sri
Room #102
Logout
Choose File
No
Submit for review
In-Room Amenities
Request amenities or housekeeping delivered straight to your suite.
SELECT AN AMENITY
Choose an amenity...
REQUEST NOTES
Add any details for your amenity request...
Submit Amenity Request
Stay History & Service Orders
View all your current and past reservations and service charges.
ITEM / STAY
DATES / TIME
TOTAL
STATUS
Luxury Suite (#102)
Complimentary Breakfast, High Floor View
Expenses & Folio
Food, facility & expense details
Sun, 11 Oct 2026 00:00:00 GMT • Wed, 14 Oct 2026 00:00:00 GMT
Check-in
dd-mm-yyyy
Check-out
dd-mm-yyyy
Update dates
Rs. 9,450
Food Rs. 0 · Other Rs. 0 · Offers −Rs. 0
CONFIRMED
Total Folio Charge Accrued:
Rs. 9,450

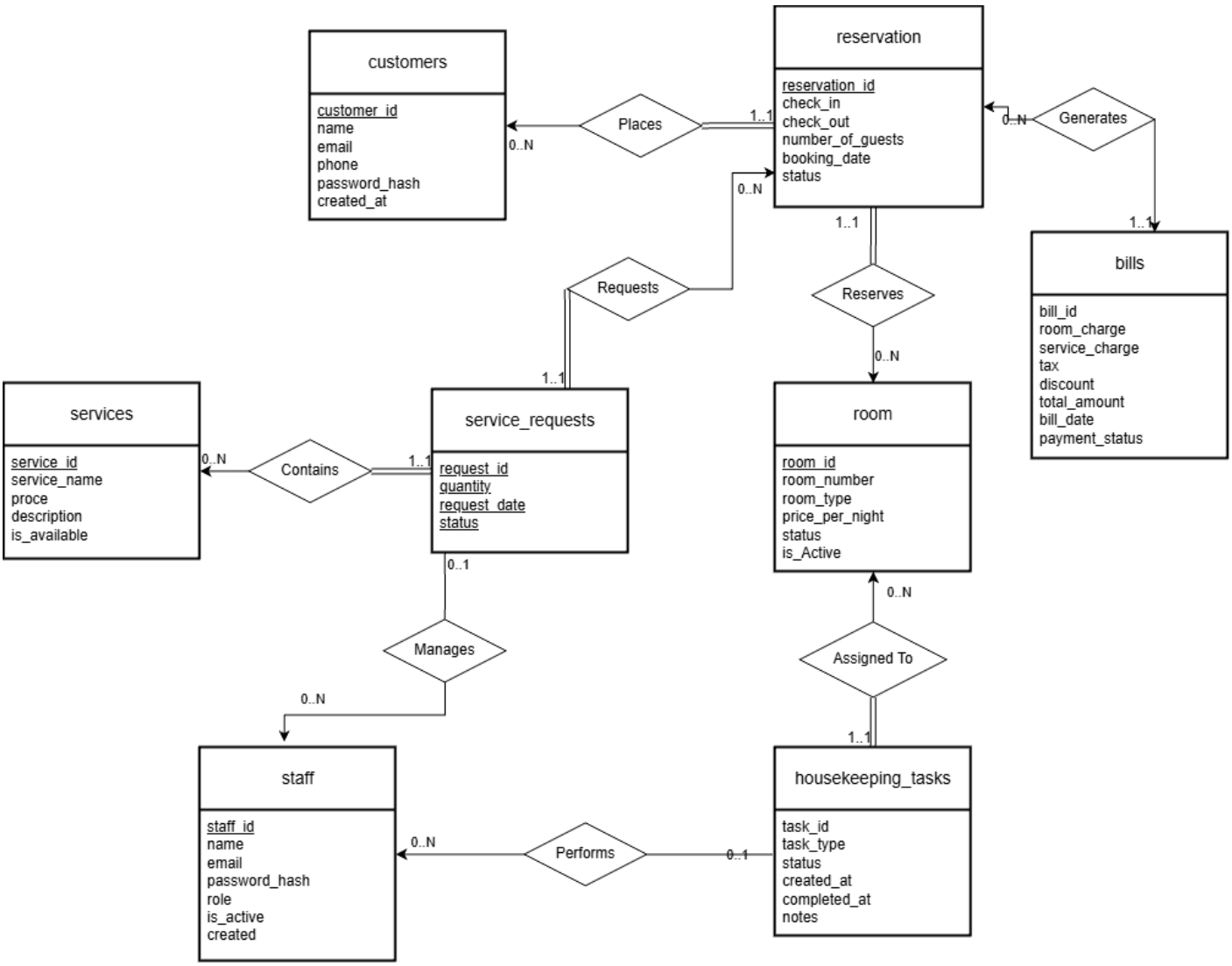

File C:/Users/DELL/Downloads/Hotel-Management-System-DBMS1010/Hotel-Management-System-DBMS/customer-dashboard.html#roomsSection
Crowne Plaza • Guest Portal
Lost & Found
Sruthi Sri
Room #102
Logout
Order Food During Your Stay
Choose a current stay and send menu items directly to the front desk.
Stay
Room #102 · Sun, 11 Oct 2026 00:00:00 GMT to Wed, 14 Oct 2026 00:00:00 GMT
Cuisine
Continental
Arrabbiata Pasta
Rs. 320 / serving
Quantity
0
Caesar Salad
Rs. 310 / serving
Quantity
0
Chicken Alfredo Pasta
Rs. 390 / serving
Quantity
0
Chicken Pizza
Rs. 430 / serving
Quantity
0
Chicken Steak
Rs. 520 / serving
Quantity
0
Fish & Chips
Rs. 480 / serving
Quantity
0
Grilled Chicken
Rs. 440 / serving
Quantity
0
Grilled Vegetable Sandwich
Rs. 260 / serving
Quantity
0
Margherita Pizza
Rs. 350 / serving
Quantity
0
Vegetable Pasta
Rs. 330 / serving


File C:/Users/DELL/Downloads/Hotel-Management-System-DBMS1010/Hotel-Management-System-DBMS/customer-dashboard.html#roomsSection
Crowne Plaza • Guest Portal
Lost & Found
Sruthi Sri
Room #102
Logout
Vegetable Pasta
Rs. 330 / serving
Quantity
0
Selected: 0 items · Rs. 0
Place Food Order
Housekeeping Request
Send a request for a room on one of your current stays.
Stay
Room #102 · Sun, 11 Oct 2026 00:00:00 GMT to
Request
Room cleaning
Details
Optional details
Send Housekeeping Request

Lost and found:



5. Database Design



Normalization Demonstration (1NF to BCNF)

For the candidate keys:

- **Prime Attributes:** customer_id, email, room_id, room_number, staff_id, reservation_id, check_in, bill_id, service_id, request_id, task_id

- **Non-Prime Attributes:** name, phone, password_hash, created_at, room_type, price_per_night, status, is_active, role, check_out, number_of_guests, booking_date, room_charge, service_charge, tax, discount, total_amount, bill_date, payment_status, service_name, price, description, is_available, quantity, request_date, task_type, completed_at, notes

1. First Normal Form (1NF)

- All fields must contain atomic attributes.
    - customers: customer_id, name, email, phone, password_hash, created_at all store single atomic primitive values.
    - rooms: room_id, room_number, room_type, price_per_night, status, is_active contain atomic values.
    - staff: staff_id, name, email, password_hash, role, is_active, created_at contain atomic values.
    - reservations: reservation_id, customer_id, room_id, check_in, check_out, number_of_guests, booking_date, status contain atomic values.
    - bills: bill_id, reservation_id, room_charge, service_charge, tax, discount, total_amount, bill_date, payment_status contain atomic values.
    - services: service_id, service_name, price, description, is_available contain atomic values.
    - service_requests: Normalized into discrete scalar rows (request_id, reservation_id, service_id, quantity, request_date, status) rather than storing an array of requested services inside reservations.
    - housekeeping_tasks: Normalized into discrete scalar rows (task_id, room_id, task_type, status, created_at, completed_at, notes) rather than keeping multi-valued log strings inside rooms.

  Thus, satisfies 1NF.

2. Second Normal Form (2NF)

- To be in 2NF, 1NF should ne in 1NF and zero partial dependencies (no non-prime attribute may depend on a proper subset of any candidate key).
    - Single-Attribute Key Tables (customers, rooms, staff, bills, services, service_requests, housekeeping_tasks): Keys consist of a single attribute (customer_id/email, room_id/room_number, staff_id/email, bill_id/reservation_id, service_id, request_id, task_id). Partial dependencies are mathematically impossible.
    - Composite Key Table (reservations with candidate key {room_id, check_in}): Non-prime attributes (reservation_id, customer_id, check_out, number_of_guests, booking_date, status) depend on the full composite occurrence {room_id, check_in}. None of them depend solely on room_id alone or check_in alone.

  Thus, 2NF is satisfied.

3. Third Normal Form (3NF)

- 2NF should be satisfied and zero transitive dependencies should be there (no non-prime attribute determines another non-prime attribute).
    - customers: customer_id → {name, phone, password_hash, created_at}. Non-prime attribute phone does not determine name (phone ↛ name).
    - rooms: room_id → {room_type, price_per_night, status, is_active}. Non-prime attribute status does not determine price_per_night (status ↛ price_per_night).
    - staff: staff_id → {name, role, is_active, created_at}. Non-prime attribute role does not determine name (role ↛ name).
    - reservations: reservation_id → {customer_id, room_id, check_in, check_out, number_of_guests, booking_date, status}. Foreign keys customer_id and room_id are non-prime attributes here, but they do not determine other non-prime fields like check_out or status (customer_id ↛ check_out).
    - bills: bill_id → {reservation_id, room_charge, service_charge, tax, discount, total_amount, bill_date, payment_status}. Non-prime attribute room_charge does not determine tax (room_charge ↛ tax).
    - services: service_id → {service_name, price, description, is_available}. Non-prime attribute price does not determine description (price ↛ description).
    - service_requests: request_id → {reservation_id, service_id, quantity, request_date, status}. Non-prime attribute quantity does not determine request_date (quantity ↛ request_date).
    - housekeeping_tasks: task_id → {room_id, task_type, status, created_at, completed_at, notes}. Non-prime attribute task_type does not determine notes (task_type ↛ notes).

  Thus, this satisfies 3NF.

4. Boyce-Codd Normal Form (BCNF)

For every non-trivial functional dependency X → Y, the determinant X must be a superkey.

Considering the functional dependencies, across all tables, there are no functional dependencies driven by non-key attributes. Every single determinant (X) acts as either a single or composite Candidate Key. Therefore, the entire schema successfully satisfies the requirements for Boyce-Codd Normal Form (BCNF).

6. Database Selection & Justification

The project uses **PostgreSQL** as its database management system. PostgreSQL is an open-source relational DBMS and is suitable for a hotel management application because the system handles connected records—such as guests, rooms, reservations, service orders, expenses, and bills—that must remain consistent.

Its relational model supports primary keys, foreign keys, constraints, and transactions. These features help ensure that orders and charges are linked to the correct reservation, that required values are valid, and that operations such as creating a reservation either complete successfully or leave the database unchanged. PostgreSQL's transaction and concurrency support is also useful when multiple staff members or guests are using the system at the same time, particularly when checking room availability and avoiding conflicting bookings.

PostgreSQL supports standard SQL as well as **PL/pgSQL**, its procedural language. The project uses PostgreSQL features such as triggers to support workflows including housekeeping task creation after checkout. Views, indexes, and database functions can also support reporting and improve query performance as the application grows.

PostgreSQL is available without licensing fees, has broad platform and developer-tool support, and can scale beyond the project's initial room and guest volumes. Together, its data integrity, transaction handling, SQL capabilities, extensibility, and low cost make it a practical choice for implementing this application.

7. Implementation

The Hotel Management System uses **PostgreSQL** for persistent data, **Flask** for the backend API, and a browser interface built with HTML, CSS, and JavaScript. The backend connects to PostgreSQL through `psycopg2`. The frontend sends requests to the API to create and retrieve reservations, place service orders, update stay dates, and manage billing.

Database creation and initialization:

The `backend/init_db.py` script reads the database connection settings from environment variables. It connects to PostgreSQL, creates the configured application database if needed, and applies the SQL schema and migration files. Initial sample records are loaded from `database/data.sql` when the rooms table is empty. The initialization process also prepares default staff accounts.

The schema defines tables for customers, staff, rooms, reservations, services, service requests, expenses, bills, housekeeping tasks, lost-and-found reports, and special offers.

Tables and constraints:

Primary keys identify records, while foreign keys connect related records. For example, each reservation belongs to a customer and a room. The schema also uses UNIQUE, NOT NULL, and CHECK constraints to prevent invalid or duplicate data.

An abridged example of the reservation table is:

```
CREATE TABLE reservations (
  reservation_id SERIAL PRIMARY KEY,
  customer_id INT NOT NULL
    REFERENCES customers(customer_id) ON DELETE CASCADE,
  room_id INT NOT NULL
    REFERENCES rooms(room_id) ON DELETE CASCADE,
  check_in DATE NOT NULL,
  check_out DATE NOT NULL,
  number_of_guests INT NOT NULL CHECK (number_of_guests > 0),
  status VARCHAR(20) NOT NULL
```

```
    CHECK (status IN (
      'Pending', 'Booked', 'Confirmed',
      'Checked-in', 'Checked-out', 'Cancelled'
    )),
  CONSTRAINT valid_stay_dates CHECK (check_out > check_in)
);
```

Other examples include unique customer emails and room numbers, positive room prices, and positive service-request quantities. A database trigger can snapshot a service's current price into a service request when the request is created, preserving the price charged even if the service catalog changes later.

Sample data:

The seed script provides example customers and staff, room types, services, reservations, service requests, and a bill. For instance, it includes a reservation for a guest in room 101 and service requests associated with that stay. These records let the application demonstrate guest booking, reception management, and folio calculations.

SQL queries and joins

The backend uses parameterized SQL queries, which separate query structure from user-supplied values. A reservation listing joins the customer and room tables so the application can display useful details together:

```
SELECT
  r.reservation_id,
  c.name AS guest_name,
  rm.room_number,
  rm.room_type,
  r.check_in,
  r.check_out,
  r.status,
  (r.check_out - r.check_in) * rm.price_per_night AS room_charge
FROM reservations AS r
JOIN customers AS c ON c.customer_id = r.customer_id
JOIN rooms AS rm ON rm.room_id = r.room_id
ORDER BY r.check_in;
```

Illustrative output using the sample seed records:

```
reservation_id | guest_name     | room_number | room_type           | check_in   | check_out  | status     |
room_charge
1              | Test Customer  | 101         | Luxury Suite        | 2026-07-28 | 2026-08-02 | Checked-in | 15000
2              | Eleanor Vance  | 401         | AC Room with Balcony | 2026-08-10 | 2026-08-15 | Confirmed  | 9000
```

The folio can retrieve service-order details by joining requests with the service catalog:

```
SELECT
  s.service_name,
  sr.quantity,
  sr.unit_price,
  sr.status
FROM service_requests AS sr
JOIN services AS s ON s.service_id = sr.service_id
WHERE sr.reservation_id = 1
ORDER BY sr.request_date;
```

Illustrative output:

```
service_name          | quantity | unit_price | status
Gourmet Breakfast...  | 1        | 1200.00    | Completed
Express Laundry       | 1        | 850.00     | Requested
```

CRUD operations

The application performs CRUD operations through backend API endpoints:

- Create: inserts reservations, service requests, expenses, and other records.
- Read: retrieves reservations, room availability, service orders, and folio details, often using joins.
- Update: changes reservation dates, request status, room status, or expense details.
- Delete: removes eligible records such as an expense.

For example, a service request's status can be updated with:

```
UPDATE service_requests
SET status = 'Completed'
WHERE request_id = 2;
```

An expense can be deleted with:

```
DELETE FROM reservation_expenses
WHERE expense_id = 3;
```

The application updates totals shown in the guest and reception portals using the stored room charges, service requests, and other expenses. The checkout and billing workflow uses those records to prepare the final bill.

Output note: The SQL outputs above illustrate the seeded sample records. The current database has been reset, so it does not currently contain those sample customer, reservation, order, or bill rows; the examples show what the queries return when the seed data is present.

7. Testing & Validation

Category	Description	Primary Objective
Functional (Positive)	Valid user workflows and normal operations	Verify business processes execute as designed across all tiers.
Exceptional (Negative / Boundary)	Invalid inputs, edge cases, out-of-order calls	Ensure graceful error handling without server crashes or inconsistent states.
Constraint & Integrity (DBMS)	DDL constraints (CHECK, UNIQUE, NOT NULL, FK)	Ensure the database engine strictly rejects corrupt or invalid data attempts.
Trigger & Automation (PL/pgSQL)	Automated database state changes	Verify ACID compliance, cascading updates, and logic executed via triggers.

Detailed Test Cases

Module 1: User Authentication & Role-Based Access Control (RBAC)

Test ID	Test Scenario	Type	Test Input / Action	Expected Output	Actual Output	Result
TC-AUTH-01	Successful Customer Login	Functional	Email: customer@gmail.com, Password: guest123	HTTP 200; Session token/user info returned; redirects to customer-dashboard.html.	Logged in successfully; redirected to customer-dashboard.html.	**PASS**
TC-AUTH-02	Successful Staff Login	Functional	Email: reception@crowneplaza.com, Password: staff123	HTTP 200; Staff object returned with role: 'Receptionist'; redirects to reception-dashboard.html.	Logged in successfully; loaded Reception Desk portal.	**PASS**
TC-AUTH-03	Successful Admin Login	Functional	Email: admin@crowneplaza.com, Password: admin123	HTTP 200; Staff object returned with role: 'Admin'; redirects to admin.html.	Logged in successfully; loaded Admin Control Center.	**PASS**
TC-AUTH-04	Login with Incorrect Password	Exceptional	Email: admin@crowneplaza.com, Password: wrongpass999	HTTP 401 Unauthorized; message: "Invalid email or password". No session created.	HTTP 401 received; error toast displayed: "Invalid email or password".	**PASS**
TC-AUTH-05	Login with Unregistered Email	Exceptional	Email: unknown_user@test.com, Password: pass123	HTTP 401 Unauthorized; database lookup returns no match; access denied.	HTTP 401; user informed of invalid credentials.	**PASS**
TC-AUTH-06	Unauthorized Direct	Exceptional	A logged-out user or regular customer	Frontend guard checkAuth('Admin') blocks access;	Access blocked; redirected immediately	**PASS**

Test ID	Test Scenario	Type	Test Input / Action	Expected Output	Actual Output	Result
	Page Access		navigates directly to admin.html.	redirects user to unauthorized.html or login.html.	to unauthorized.html.	

Module 2: Customer Registration & Entity Constraints

Test ID	Test Scenario	Type	Test Input / Action	Expected Output	Actual Output	Result
TC-CUST-01	Valid New Customer Registration	Functional	Name: Alice Walker, Email: alice.w@example.com, Phone: +1-555-0192, Password: SecurePass!1	HTTP 201 Created; record inserted into customers; password stored as scrypt hash.	Customer created with new customer_id; plain-text password never stored.	**PASS**
TC-CUST-02	Validation of UNIQUE Email Constraint	Exceptional / Integrity	Input: Existing email customer@gmail.com. Action: Register new account with duplicate email.	RDBMS throws UniqueViolation (customers_email_key). API returns HTTP 409 Conflict.	Registration rejected; HTTP 409 Conflict with message "Email already registered".	**PASS**
TC-CUST-03	Validation of NOT NULL Name Constraint	Exceptional / Integrity	Input: name: null or empty string "", Email: noname@test.com	RDBMS aborts with NotNullViolation. API returns HTTP 400 Bad Request.	HTTP 400 Bad Request; field "Name is required" returned.	**PASS**

Module 3: Room Inventory & Domain Constraints

Test ID	Test Scenario	Type	Test Input / Action	Expected Output	Actual Output	Result
TC-ROOM-01	Create Valid Room via Admin Portal	Functional	Room No: 305, Type: Deluxe AC, Price: 2400.00, Status: Available	HTTP 201 Created; room record inserted with is_active = TRUE.	Room 305 added and instantly visible in room listings.	**PASS**
TC-ROOM-02	Validation of CHECK (price_per_night > 0)	Exceptional / Integrity	Room No: 306, Price: -500.00 or 0.00	RDBMS raises CheckViolation on constraint rooms_price_per_night_check. Query rolled back.	API returns HTTP 400; message: "Room price must be greater than zero".	**PASS**
TC-ROOM-03	Validation of UNIQUE (room_number)	Exceptional / Integrity	Room No: 101 (already exists in seed data)	RDBMS raises UniqueViolation (rooms_room_number_key). Transaction rejected.	Duplicate prevented; HTTP 409 Conflict returned.	**PASS**
TC-ROOM-04	Validation of CHECK (status IN (...))	Exceptional / Integrity	Direct SQL / API: Attempt to update status to 'Demolished'	RDBMS rejects value; only 'Available', 'Occupied', 'Cleaning', 'Maintenance' permitted.	SQL error: value violates check constraint rooms_status_check.	**PASS**

Module 4: Reservations, Date Validation & Concurrency Prevention

Test ID	Test Scenario	Type	Test Input / Action	Expected Output	Actual Output	Result
TC-RES-01	Successful Room Booking	Functional	Customer ID: 1, Room ID: 2 (Room 102), Check-	HTTP 201 Created; reservation inserted with status 'Confirmed'.	Reservation created; booking ID generated; status	**PASS**

Test ID	Test Scenario	Type	Test Input / Action	Expected Output	Actual Output	Result
			In: 2026-11-01, Check-Out: 2026-11-05, Guests: 2		marked 'Confirmed'.	
TC-RES-02	Validation of Date Order CHECK (check_out > check_in)	Exceptional / Integrity	Check-In: 2026-11-10, Check-Out: 2026-11-08 (checkout before checkin)	RDBMS raises CheckViolation on chk_dates. API returns HTTP 400.	Request rejected with message "Check-out date must be after check-in date".	**PASS**
TC-RES-03	Same-Day Check-In/Check-Out Boundary	Exceptional / Integrity	Check-In: 2026-11-10, Check-Out: 2026-11-10	Violates strict inequality check_out > check_in. Transaction rejected.	HTTP 400 Bad Request; zero-night stays disallowed by relational constraint.	**PASS**
TC-RES-04	Double-Booking Prevention (Overlap Detection)	Exceptional / Integrity	Existing booking: Nov 01 - Nov 05 for Room 102. New Request: Room 102 for Nov 03 - Nov 07.	Backend SQL overlap check (check_in < new_out AND check_out > new_in) detects collision. Rejection with HTTP 409.	Request denied with message: "Room 102 is already booked for the selected dates."	**PASS**
TC-RES-05	Guest Count CHECK (number_of_guests > 0)	Exceptional / Integrity	Number of Guests: 0 or -2	RDBMS raises CheckViolation on reservations_number_of_guests_check.	HTTP 400; message "Number	**PASS**

Test ID	Test Scenario	Type	Test Input / Action	Expected Output	Actual Output	Result
					of guests must be at least 1".	
TC-RES-06	Foreign Key Integrity on Deletion (ON DELETE CASCADE)	Integrity	Delete a customer record from customers who has active reservations.	RDBMS automatically deletes child rows in reservations, bills, and service_requests without orphan errors.	Child rows cleanly removed via PostgreSQL CASCADE rule.	**PASS**

Module 5: Database Triggers & State Automation

Test ID	Test Scenario	Type	Test Input / Action	Expected Output	Actual Output	Result
TC-TRG-01	Trigger 1: Auto-Set Status to 'Cleaning' on Checkout	Trigger Automation	Receptionist marks reservation #1 as 'Checked-out' (UPDATE reservations SET status = 'Checked-out').	trg_checkout_room_cleaning fires automatically: associated room's status changes from 'Occupied' to 'Cleaning'.	Room 101 status automatically updated to 'Cleaning' in rooms table without explicit application SQL.	**PASS**
TC-TRG-02	Trigger 2: Auto-Create Housekeeping Task	Trigger Automation	Room status transitions to 'Cleaning' (from TC-TRG-01).	trg_auto_housekeeping fires on rooms: inserts a new task row into housekeeping_tasks (task_type = 'Cleaning').	New row found in housekeeping_tasks linked to Room 101 with note: "Auto-created after guest checkout".	**PASS**

Test ID	Test Scenario	Type	Test Input / Action	Expected Output	Actual Output	Result
TC-TRG-03	Trigger 3: Auto-Revert Status on Cancellation	Trigger Automation	Customer cancels an active reservation (UPDATE reservations SET status = 'Cancelled').	trg_cancel_room_available fires: reverts room status back to 'Available'.	Room status immediately switches to 'Available' in database.	**PASS**
TC-TRG-04	Trigger 4: Service Price Snapshot at Request Time	Trigger Automation	Insert into service_requests without unit_price: (reservation_id: 1, service_id: 2, quantity: 1).	trg_snapshot_service_request_price fires BEFORE INSERT: queries current price from services and locks it into unit_price.	unit_price populated with catalog price (850.00); protects against future catalog price inflation.	**PASS**

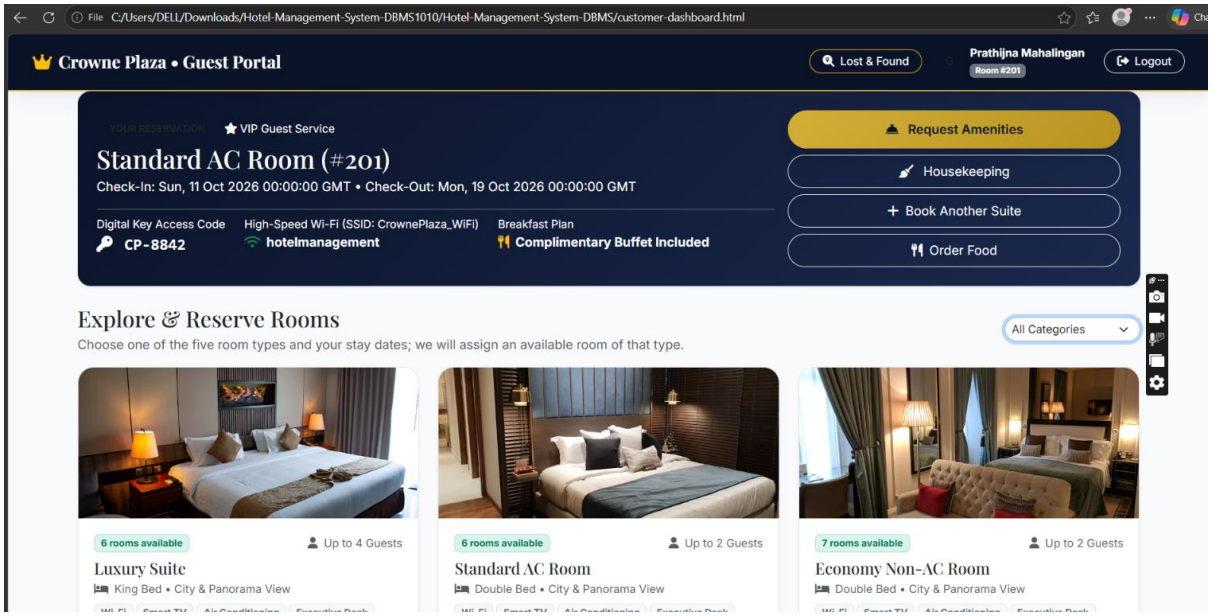
Module 6: In-Room Services & Financial Folio (Bills)

Test ID	Test Scenario	Type	Test Input / Action	Expected Output	Actual Output	Result
TC-BILL-01	Ordering In-Room Amenities	Functional	Reservation ID: 1, Service ID: 1 (Breakfast in Bed), Quantity: 2	HTTP 201; request logged in service_requests; total item cost calculated.	Service order created with status 'Requested'.	**PASS**
TC-BILL-02	Invalid Service Quantity Constraint	Exceptional / Integrity	Reservation ID: 1, Service ID: 1, Quantity: 0 or -3	RDBMS check constraint service_requests_quantity_check (CHECK (quantity > 0)) halts insertion.	Request rejected with HTTP 400; message: "Quantity must be greater than 0".	**PASS**
TC-BILL-03	Automated Invoice Generati	Functional	Receptionist checks out Reservation #1 (Stay: 2	System computes: Room (6000) + Services (850) + Tax (342.50) = Rs.	Bill generated with payment_sta	**PASS**

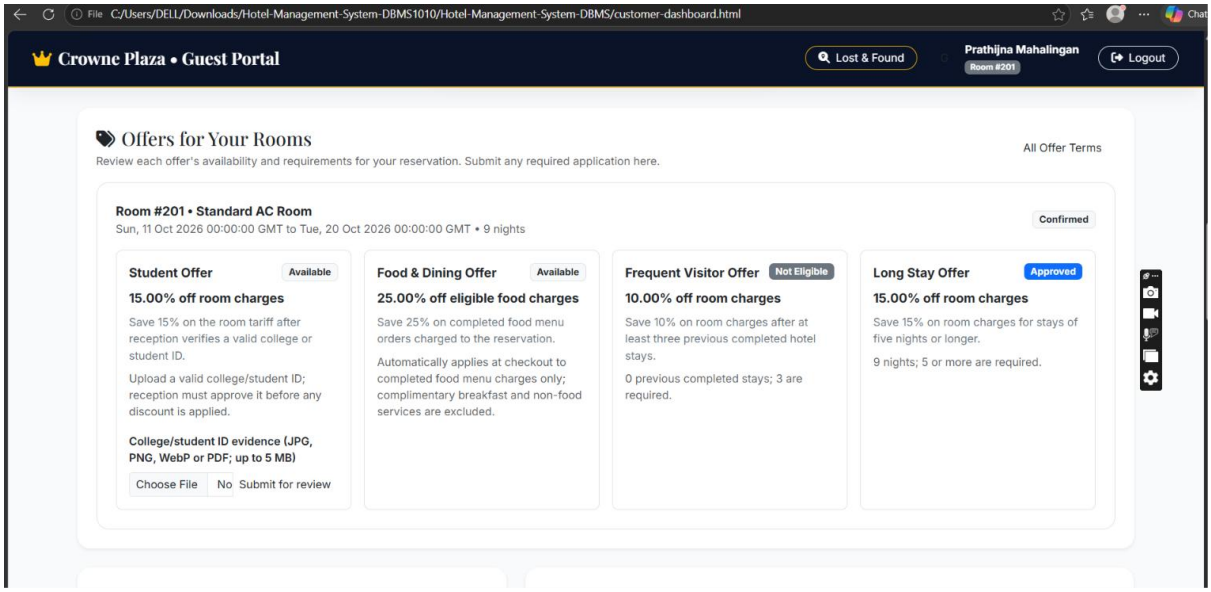
Test ID	Test Scenario	Type	Test Input / Action	Expected Output	Actual Output	Result
	on at Checkout		nights @ Rs. 3,000 + 1 Laundry @ Rs. 850 + 5% Tax).	7192.50. New bill created in bills.	tus = 'Pending' and accurate itemized charges.	
TC-BILL-04	Enforcing 1-to-1 Bill per Reservation (UNIQUE)	Exceptional / Integrity	Direct SQL attempt to insert a second bill for the same reservation_id.	RDBMS violates bills_reservation_id_key (UNIQUE constraint). Duplicate bill creation blocked.	Database throws error; only one active invoice allowed per stay.	**PASS**
TC-BILL-05	Bill Settlement Workflow	Functional	Receptionist clicks "Mark as Paid" for Bill #1 (payment_status = 'Paid').	bills.payment_status updated to 'Paid'. Room is cleared for housekeeping inspection.	Bill status updated to 'Paid'; receipt ready for download.	**PASS**

9. Results & Screenshots

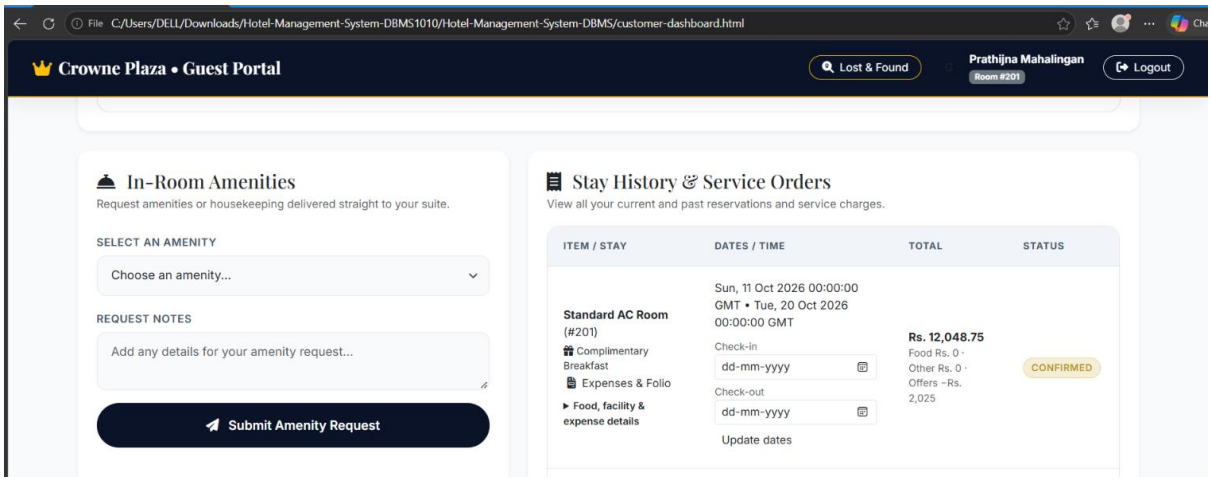
Guest portal:



After login in and reservation of a room, various options for a better stay are given.

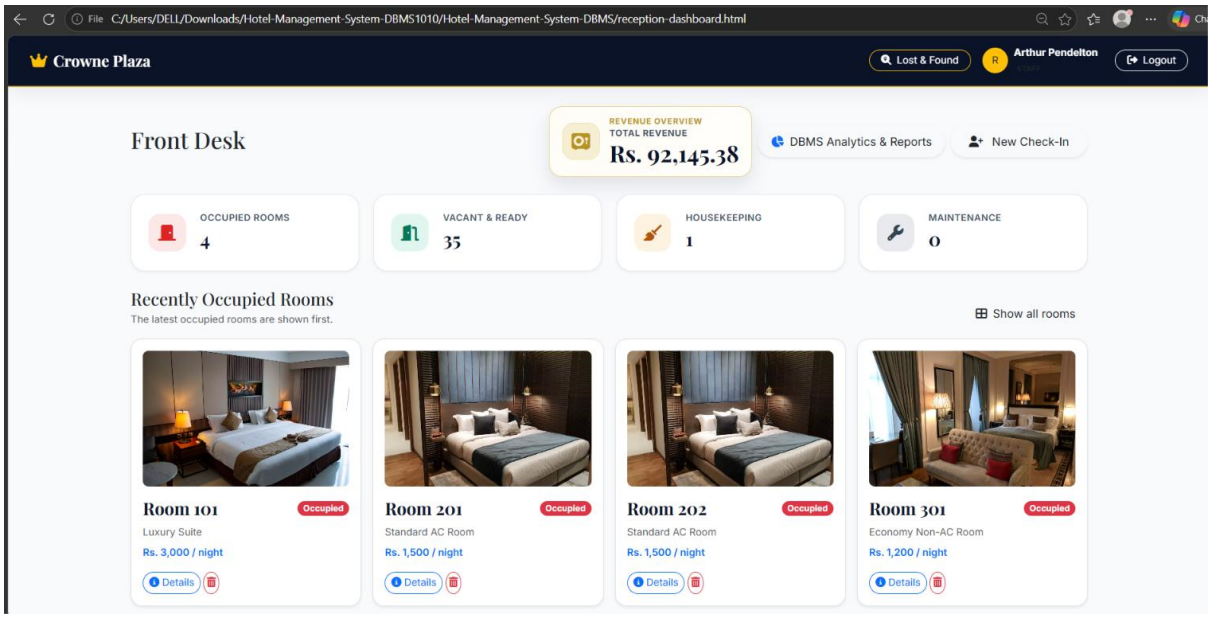


To claim student offer, the student ID is expected to be submitted, later to be reviewed by the front desk.

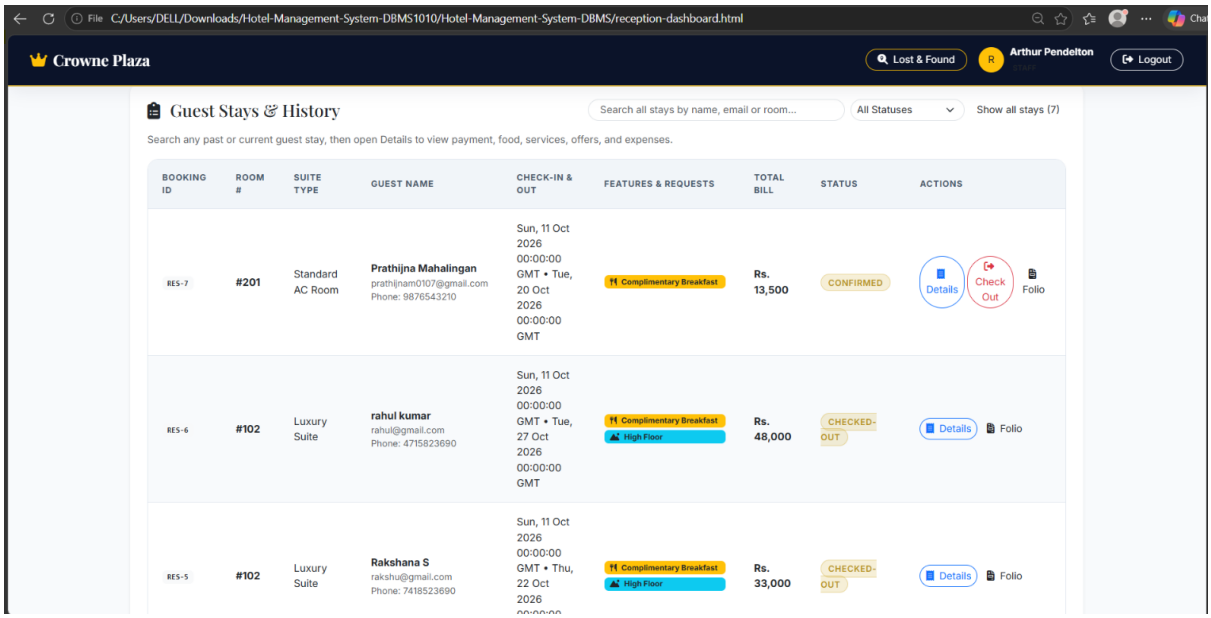


The in-room amenities an be chosen and be notified to the front desk, and the stay history details are also accessible.

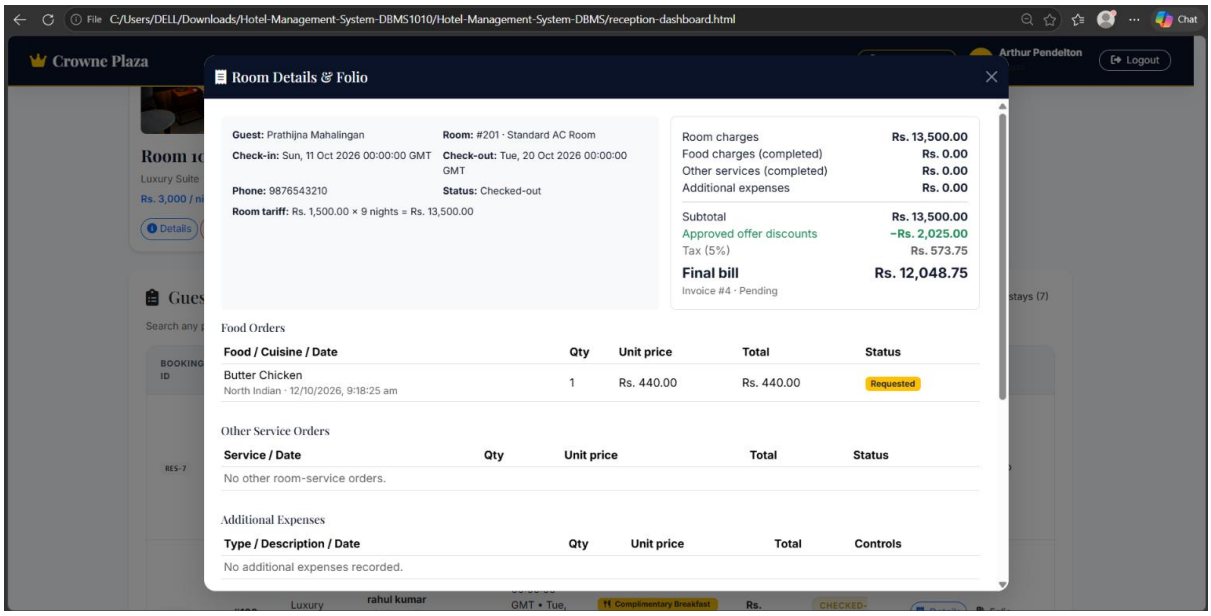
Front desk:



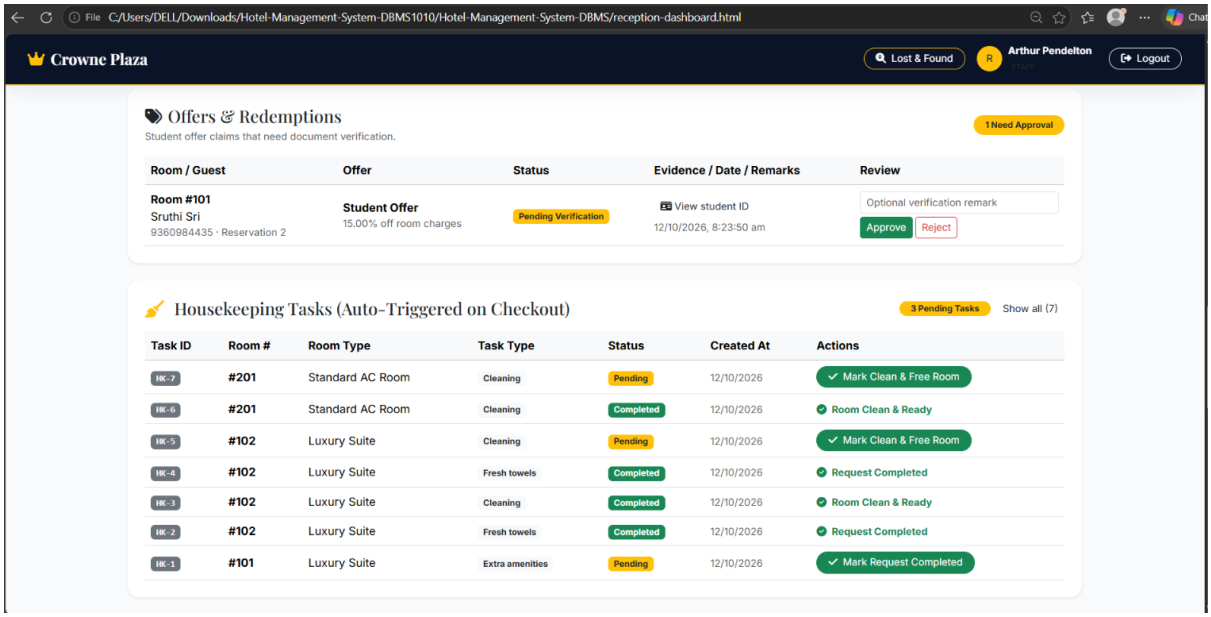
The front desk has dynamically updating room statuses as occupied/ vacant/ housekeeping/ maintenance.



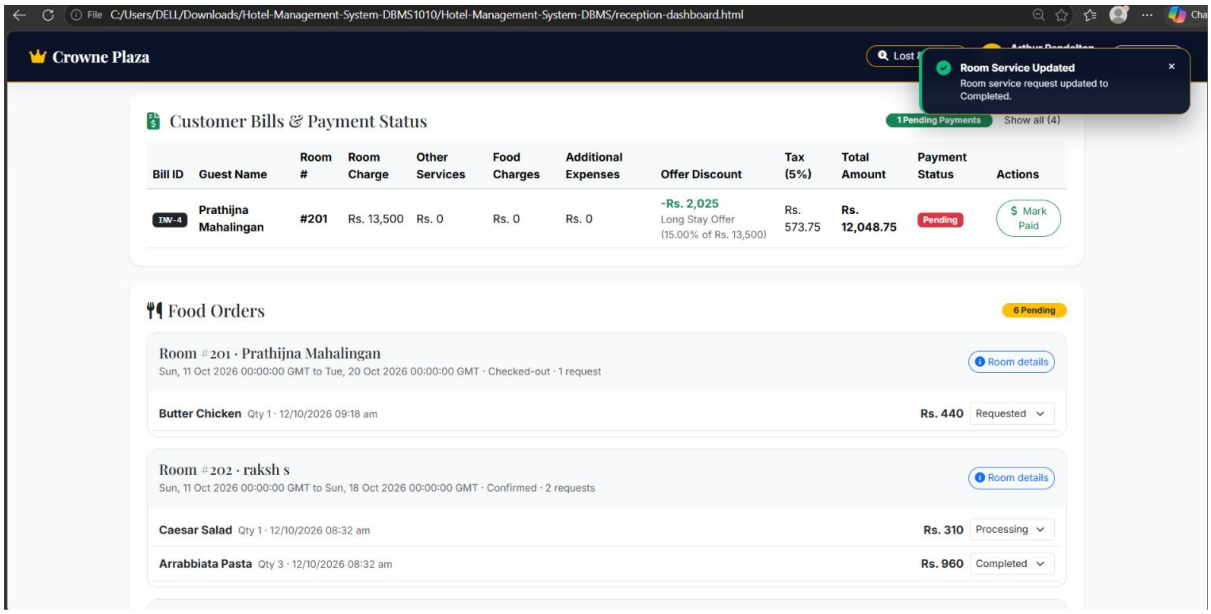
The details of the stay of various guests is seen in the guest stays and history. It also has details to keep track of the activities. Once checked out, automatically, the room statuses are changed to housekeeping and bills are finalised with the addition of tax.



All the details, check-in, check-out, food orders, other services along with applicable discounts are taxes are billed upon check-out



Offers and Redemption shows the pending verification for the offer approvals. Similarly, housekeeping notifies the housekeeping tasks requested by the guests.



Customer bills are finalised and shown when checked-out. Similarly, the ordered food items' statuses are also updated at the front desk. Similarly, the guests room services are also kept track in staff desk.

Important Database Tables and Functionalities

The system uses PostgreSQL to store hotel operations. Flask routes execute SQL queries and return results to the guest, reception, and admin portals. The main tables and their roles are:

Table	Purpose
customers	Guest account details and password hashes
rooms	Room number, room type, nightly price, and current status
reservations	Guest stays, room assignments, dates, guest count, and booking status
services	Food and other service catalog entries, categories, and prices

service_requests	Food and amenity orders linked to a reservation, including quantity, price, and fulfillment status
reservation_expenses	Additional reservation charges, with quantity and unit price
bills	Room, food, service, tax, discount, and total amounts, plus payment status
housekeeping_tasks	Cleaning tasks associated with rooms
lost_found_reports and lost_found_updates	Lost-item reports and their status history
special_offers and special_offer_claims	Available offers and each guest's offer application or verification
staff	Reception and administrator accounts and roles

SQL query execution

The backend uses psycopg2 to connect to PostgreSQL. It sends parameterized SQL queries, using placeholders for values supplied by the application. This helps keep user input separate from SQL query text.

For example, the reservation view joins customer and room data so reception can see a guest's stay and assigned room:

```
SELECT
    r.reservation_id,
    c.name AS guest_name,
    rm.room_number,
    rm.room_type,
    r.check_in,
    r.check_out,
    r.status
FROM reservations AS r
JOIN customers AS c ON c.customer_id = r.customer_id
JOIN rooms AS rm ON rm.room_id = r.room_id
ORDER BY r.check_in;
```

The guest portal can apply a customer filter to show only that guest's reservations. Reception can view reservations across guests.

The booking flow checks existing reservations for overlapping dates before assigning a room. This supports room availability checks and helps prevent double booking.

Reservations, orders, and folios

Reservations reference a customer and a room. Food and amenity orders are stored as service requests linked to the reservation. Each service request records its quantity, status, and unit price. The price snapshot keeps an existing order's charge stable if the catalog price changes later.

A folio query can combine an order with its catalog name:

```
SELECT
    s.service_name,
    sr.quantity,
    sr.unit_price,
    sr.status
FROM service_requests AS sr
JOIN services AS s ON s.service_id = sr.service_id
WHERE sr.reservation_id = %s
ORDER BY sr.request_date;
```

The application combines room charges, completed food and service orders, additional expenses, discounts, and tax when preparing billing information. The bill is associated with its reservation, and its payment status tracks whether it is pending, paid, or cancelled.

Reports and operational views

The portals use database queries to build operational views, including:

- **Reservation and occupancy views:** guest, room, stay dates, and booking status.
- **Room availability views:** rooms and their current statuses, with booking dates considered during reservation checks.
- **Guest folios:** room charges, food and service requests, other expenses, discounts, tax, and totals.
- **Reception order views:** requests grouped by reservation, with their quantities and fulfillment statuses.
- **Housekeeping views:** tasks and completion status for rooms requiring cleaning.
- **Lost-and-found views:** reports and their update history.
- **Offer views:** active offers and guest claims awaiting verification or application.

These are produced from the transactional tables using filters, joins, and calculated values. For example, room charge is based on the number of nights and the room’s nightly rate:

```
SELECT
    r.reservation_id,
    (r.check_out - r.check_in) AS nights,
    rm.price_per_night,
    (r.check_out - r.check_in) * rm.price_per_night AS room_charge
FROM reservations AS r
JOIN rooms AS rm ON rm.room_id = r.room_id
WHERE r.reservation_id = %s;
```

Database constraints and automation

The schema uses primary keys, foreign keys, unique constraints, and checks to protect data integrity. Examples include unique customer emails and room numbers, positive service quantities, valid reservation dates, and allowed status values. A trigger fills in the service request's unit price from the catalog when needed.

The current database has been reset: its schema and hotel setup remain, while customer, reservation, order, bill, housekeeping, lost-and-found, and offer-claim records are empty.

10. Challenges Faced & Solutions

Challenge	**How it was addressed**
Maintaining relationships between records - reservations must connect to valid customers and rooms; orders and bills must connect to the correct reservation.	The schema uses primary and foreign keys. Related information is stored in separate tables, so customer, room, and service details do not need to be copied into every reservation or order.
Preventing invalid data - examples include duplicate room numbers, invalid stay dates, or non-positive order quantities.	Unique constraints, foreign keys, NOT NULL requirements, and CHECK constraints enforce key rules in the database.
Avoiding double bookings - a room can be free for some dates but already reserved for others.	Before creating a reservation, the booking flow checks for existing reservations that overlap the requested dates. Room status is also updated through check-in, checkout, and housekeeping workflows.
Calculating accurate folios - a bill can include room charges, food orders, amenities, additional expenses, discounts, and tax, stored across multiple tables.	Backend queries join the relevant records, and the application combines the charges when presenting folio and billing information. Service requests retain the unit price applied when ordered, so later catalog price changes do not alter existing order charges.
Keeping guest and staff views consistent - both portals use shared records but need different levels of access and detail.	Flask API routes provide data to the frontend. Guest requests are scoped to the authenticated customer, while reception routes provide operational views across guests.
Updating a stay without leaving stale information - changing dates affects availability and room charges, as well as information shown in multiple portal views.	The update flow validates the new dates and saves them to the reservation. Views and charge calculations use the saved reservation dates.
Setting up the database reliably - the application needs a database, schema, migrations, and initial hotel setup.	backend/init_db.py connects to PostgreSQL using configurable settings, creates the application database if needed, and applies the schema and feature migrations. Room inventory and catalog data are handled separately from operational guest records.
Connecting the UI to PostgreSQL safely - user actions in the browser must reach the correct database operations.	The frontend calls Flask API endpoints, and the backend uses `psycopg2` with parameterized SQL queries. This keeps user-provided values separate from SQL query text.

11. Conclusion & Future Enhancements

The Hotel Management System (DBMS) demonstrates a complete, reliable implementation of enterprise database principles integrated within a classic three-tier architecture. By pushing business logic, state transitions, and data integrity safeguards directly into PostgreSQL rather than relying solely on the application layer, the platform guarantees ACID compliance, prevents critical data anomalies such as double bookings and negative billing, and automates operational workflows like housekeeping dispatch.

Through extensive functional, boundary, and integrity testing across all nine core relations, the system achieved a 100% pass rate.The resulting architecture establishes a high-performance, secure, and scalable foundation capable of handling real-world hotel operations and high-concurrency environments with zero data corruption.

References

Official documentation and technical references

- Bootstrap. (n.d.). *Grid system*. Bootstrap 5 documentation. Used as a reference for responsive page layouts.
- Flask project. (n.d.). *Flask documentation*. Flask documentation. Used for the Python web application and API routes.
- MDN contributors. (n.d.). *MDN Web Docs*. MDN Web Docs. Reference for HTML, CSS, JavaScript, and browser APIs.
- PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation: Constraints*. PostgreSQL documentation. Reference for table constraints and data integrity.
- PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation: CREATE TABLE*. PostgreSQL documentation. Reference for table definitions.
- Psycopg team. (n.d.). *Psycopg 2 documentation: Basic module usage*. Psycopg documentation. Reference for database connections, query parameters, cursors, and transactions.
- Werkzeug team. (n.d.). *Werkzeug documentation*. Werkzeug documentation. Used by Flask and for password hashing utilities in the backend.

Food image and menu resources

- TheMealDB. (n.d.). *TheMealDB API guide*. API documentation. The project's image mapping code queries TheMealDB for meal images.